

Ștefan-Darian CÎRNAȚ

PROIECT DE DIPLOMĂ

Anomaly Detection and Traffic Prediction with Isolation
Forest (IF) and Long Short-Term Memory (LSTM) Models

SPECIALIZAREA TEHNOLOGII ȘI SISTEME DE TELECOMUNICAȚII

Marchează dacă ai studiat
în limba engleză:



FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII
ȘI TEHNOLOGIA INFORMAȚIEI
UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
FACULTATEA DE ELECTRONICĂ
TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI
Specializarea: Tehnologii și Sisteme de Telecomunicații

Anomaly Detection and Traffic Prediction with Isolation
Forest (IF) and Long Short-Term Memory (LSTM) Models

Proiect de diplomă

Absolvent,
Ștefan-Darian CÎRNAȚ



Decan,
Prof.dr.ing. Ovidiu POP

OVIDIU-
AUREL POP

Digitally signed by
OVIDIU-AUREL POP
Date: 2025.07.14
12:52:10 +03'00'

Președinte comisie,
Prof.dr.ing. Virgil DOBROȚĂ

VIRGIL-MIRCEA
DOBROȚĂ

Digitally signed by VIRGIL-MIRCEA DOBROȚĂ
DN: cn=VIRGIL-MIRCEA DOBROȚĂ, sn=DOBROȚĂ,
givenName=VIRGIL-MIRCEA,
serialNumber=201104209VMD8,
email=virgil.dobrota@com.utcluj.ro,
o=UNIVERSITATEA TEHNICA DIN CLUJ NAPOCA,
l=CLUJ-NAPOCA, st=Cluj, c=RO, title=DIRECTOR
DEPARTAMENT
Date: 2025.07.19 20:47:14 +03'00'

2 00 12 45

Această pagină a fost lăsată goală în mod intenționat.

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
FACULTATEA DE ELECTRONICĂ
TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI

Departamentul Comunicații

Titlul proiectului de diplomă:

Anomaly Detection and Traffic Prediction with Isolation Forest (IF) and Long
Short-Term Memory (LSTM) Models

Descrierea temei:

Această diplomă va evalua posibilitatea de a prezice traficul cu model Long Short-Term Memory (LSTM) și de a detecta anomaliiile cu Isolation Forest (IF) într-o topologie de rețea implementată în GNS3 folosind containere Docker ca puncte finale pentru a genera trafic.

Locul de realizare:

UC Labs, Strada Dorobanților 71-73, sala 211/A

Adaugă
consultant ☐

Data emiterii temei: 3 0.09.2024

Data predării temei: 09.07.2025

Absolvent,

Ștefan-Darian CÎRNAȚ



Director departament,

Prof.dr.ing. Virgil DOBROTĂ

Conducător,

Prof.dr.ing. Virgil DOBROTĂ

**VIRGIL-MIRCEA
DOBROTA**

Digitally signed by VIRGIL-MIRCEA DOBROTA
DN: cn=VIRGIL-MIRCEA DOBROTA,
o=DOBROTA, givenName=VIRGIL-MIRCEA,
serialNumber=201104209VMD8,
email=virgil.dobrota@com.utcluj.ro,
o=UNIVERSITATEA TEHNICĂ DIN CLUJ NAPOCA,
l=CLUJ-NAPOCA, st=Cluj, c=RO, title=DIRECTOR
DEPARTAMENT

Date: 2025.07.14 10:30:57 +03'00'



Această pagină a fost lăsată goală în mod intenționat.

Declarație pe proprie răspundere privind autenticitatea lucrării de diplomă

Subsemnata/ul Ștefan-Darian CÎRNAȚ

legitimată cu C I seria A X nr. 773834 (conform copiei anexate prezentei declarații,
copie semnată și certificată "conform cu originalul"), autorul lucrării

Anomaly Detection and Traffic Prediction with Isolation Forest (IF) and Long Short-Term
Memory (LSTM) Models

elaborată în vederea susținerii examenului de finalizare a studiilor de licență,
la Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Programul de studiu Tehnologii și Sisteme de Telecomunicații
din cadrul Universității Tehnice din Cluj-Napoca
sesiunea i u l i e a anului universitar 2024-2025

declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele
și pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației
române și a convențiilor internaționale privind drepturile de autor. Declar, de asemenea, că această lucrare nu a mai
fost prezentată în fața unei alte comisii de examen de absolvire/licență/diplomă/disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative,
respectiv anularea examenului de diplomă.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea
autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine și conform
copiei documentului de identitate menționat mai sus.

Data

09.07.2025

Nume și prenume

Ștefan-Darian CÎRNAȚ

Semnătura



Această pagină a fost lăsată goală în mod intenționat.

Absolvent: Ștefan-Darian CÎRNAȚ
Conducător: Prof.dr.ing. Virgil DOBROTĂ

SINTEZA PROIECTU LUI DE DIPLOMĂ

Anomaly Detection and Traffic Prediction with Isolation Forest (IF) and Long Short-Term Memory (LSTM) Models

This diploma thesis presents how Artificial Intelligence is used to predict traffic (LSTM) and to detect anomalies (IF) in a basic network topology implemented in GNS3 v2.2.53 using Docker containers as endpoints to generate traffic. The virtual machine Ubuntu 20.04.6 LTS hosting the emulator ran on Windows 11, with VMware® Workstation 17.6.3 Pro. There were four Docker containers: (1) *Idle* (it does not receive or transmit traffic); (2) *Ping* (it sends ICMP request at fixed intervals); (3) *HTTP* (it uses a browser to generate HTTP); and (4) *Anomaly* (to simulate anomalous traffic, such as a ping flooding). There are two functions written in Python: (1) `fetch_multiple_metrics` handles the data extracted by Prometheus as input for the models; (2) `push_metrics` sends the AI results back to the Pushgateway, to be forwarded to Prometheus to be shown in Grafana. There were three scenarios for experiments: (a) GNS3 topology without any facilities; (b) GNS3 topology with monitoring; (c) GNS3 topology with monitoring and AI. During the very first few minutes the real-time results were not reliable nor relevant. However, the work is under progress to estimate the performance for both prediction and detection, as the thesis was focused on the design of this proposed solution.

Detectarea anomaliilor și în predicția traficului cu modelele Isolation Forest (IF) și în Long Short-Term Memory (LSTM)

Această diplomă prezintă modul în care inteligența artificială poate fi utilizată pentru a prezice traficul (cu LSTM) și pentru a detecta anomalii (cu IF) într-o topologie de rețea implementată în GNS3 v2.2.53 folosind containere Docker ca puncte finale pentru a genera trafic. Mașina virtuală Ubuntu 20.04.6 LTS a găzduit emulatorul rulant în Windows 11, cu VMware Workstation 17.6.3 Pro. Au existat patru containere Docker: (1) *Idle* (nu primește și nu transmite trafic); (2) *Ping* (trimite cereri ICMP la intervale fixe); (3) *HTTP* (folosește un browser pentru a genera HTTP); și (4) *Anomaly* (pentru a simula traficul anormal, cum ar fi inundație cu ping). Există două funcții scrise în Python: (1) `fetch_multiple_metrics` gestionează datele extrase de Prometheus ca intrare pentru modele; (2) `push_metrics` trimite rezultatele înapoi către Pushgateway, pentru a fi transmise către Prometheus și afișate în Grafana. Au existat trei scenarii pentru experimente: (a) topologie GNS3 fără nicio facilități; (b) topologie GNS3 cu monitorizare; (c) topologie GNS3 cu monitorizare și inteligență artificială. În primele minute, rezultatele în timp real nu au fost nici fiabile, nici relevante. Cu toate acestea, testarea este în curs de desfășurare pentru a estima performanța, atât pentru predicție, cât și pentru detecție, deoarece teza s-a concentrat pe proiectarea software a acestei soluții propuse.

Avizul conducătorului
Conducător,



Absolvent,



Această pagină a fost lăsată goală în mod intenționat.

Table of Contents

Table of Contents	1
List of Figures	2
List of Tables	3
Abbreviations	4
Rezumat în limba română	5
1. Work Planning	12
2. State of the Art	13
3. Theoretical Fundamentals	16
3.1 Anomaly Detection	16
3.2 Traffic Prediction	17
3.3 Long Short-Term Memory Model	17
3.4 Isolation Forest Model.....	20
3.5 GNS3	22
3.6 Docker Containers	23
3.7 Monitoring Tools: Prometheus, Grafana and Pushgateway	24
3.8 SNMP and Node Exporter for Scrape Metrics	27
4. Implementation	28
4.1 GNS3 Installation with Docker Containers	28
4.2 GNS3 Network Topology	30
4.3 Installation and Configuration of Monitoring Tools	33
4.4 Integration of AI Models for Network Monitoring	37
5. Experimental Results	44
5.1 Scenario 1: GNS3 Topology without Monitoring and AI Facilities	44
5.2 Scenario 2: GNS3 Topology with Monitoring Capabilities	45
5.3 Scenario 3: Network Topology with Monitoring and AI Integration	48
6. Conclusions	52
7. References	53
8. Appendix	55
8.1 SSET 2025 Paper	55
8.2 SSET 2025 First Special Award	57
8.3 Curriculum Vitae	58

List of Figures

Figure 1. Architecture of an LSTM Unit	18
Figure 2. Isolation Forest architecture [Wen21]	20
Figure 3. An example of isolating a non-anomalous point in a 2D Gaussian distribution	21
Figure 4. An example of isolating an anomalous point in a 2D Gaussian distribution	21
Figure 5. GNS3 GUI	22
Figure 6. Simple GNS3 topology with Docker container and Cloud node [GNS25]	23
Figure 7. Docker Container architecture [Tur24]	24
Figure 8. Prometheus Interface: Query results and target status overview [Pro25]	25
Figure 9. Grafana interface	26
Figure 10. Prometheus metric collection via SNMP and Node Exporters	27
Figure 11. GNS3 network topology used for traffic monitoring and anomaly detection	30
Figure 12. Grafana dashboard	36
Figure 13. Grafana new dashboard	37
Figure 14. GUI provided by Webterm container	45
Figure 15. Prometheus query showing traffic per router interface	47
Figure 16. Visualization of network metrics in Grafana	47
Figure 17. Traffic prediction visualization in Grafana.....	48
Figure 18. MSE for Traffic prediction in Grafana	49
Figure 19. Anomaly flags visualized in Grafana dashboard	50
Figure 20. Performance metrics for anomaly detection system displayed in Grafana	51

List of Tables

Table 1. A comparison between the most common network emulators [Pyn25]	15
Table 2. Recorded prediction results, real traffic, MSE, timestamps and runtime for LSTM	50

Abbreviations

ARIMA	Auto Regressive Integrated Moving Average
CNN	Convolutional Neural Networks
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
FSM	Finite State Machine
GAN	Generative Adversarial Networks
GNS3	Graphical Network Simulator 3
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
IF	Isolation Forest
KPIs	Key Performance Indicators
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MIBs	Management Information Bases
ML	Machine Learning
MSE	Mean Absolute Error
NAT	Network Adresse Translation
OIDs	Object Identifiers
OS	Operating Systems
PromQL	Prometheus Query Language
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
SDN	Software-Defined Networking
SNMP	Simple Network Management Protocol
SVR	Support Vector Regression
UDP	User Datagram Protocol
VM	Virtual Machine

Rezumat în limba română

1. Stadiul actual

Acest capitol oferă o privire de ansamblu asupra principalelor lucrări și tehnologii utilizate în monitorizarea rețelelor, cu accent pe integrarea containerizării, a simulatoarelor de rețea și a modelelor de învățare automată. Soluții precum Docker și GNS3 sunt frecvent folosite pentru testarea și analizarea traficului, datorită flexibilității și capacității de emulare realistă a rețelelor [Pou24a], [Alb22]. Docker permite izolarea aplicațiilor și gestionarea eficientă a resurselor, iar Kubernetes extinde aceste funcționalități la scară mai largă [Pou24b]. Totuși, în scenarii educaționale sau de cercetare, Docker rămâne o opțiune potrivită. GNS3 este un mediu ideal de dezvoltare a topologiilor ce integrează foarte bine diferite dispozitive de rețea care aproximează realitatea [Tur19], [Alb22].

Integrarea inteligenței artificiale în rețele este realizată prin modele simple, precum cele bazate pe arbori de decizie. Acestea pot îmbunătăți performanța rețelelor SD-WAN în timp real [Emm23], iar articole recente arată importanța selecției caracteristicilor și a preprocesării datelor pentru o funcționare eficientă a modelului [Sal24], [Lyp24].

Prometheus este recunoscut ca o soluție robustă pentru colectarea metricilor de rețea și integrarea cu instrumente de vizualizare precum Grafana sau cu modele AI [Pro25]. Pentru analiza avansată, algoritmi precum Isolation Forest folosit în detecția de anomalii [Xia18] și rețelele LSTM [Zha22] utilizate în predicția traficului.

2. Fundamentare teoretică

Detecție de anomalii

Detectarea anomaliilor joacă un rol esențial în monitorizarea rețelelor, deoarece ajută la identificarea comportamentelor neobișnuite care pot semnaliza probleme precum erori, configurări greșite sau chiar atacuri cibernetice. Algoritmul Isolation Forest este o alegere potrivită pentru acest tip de sarcină, deoarece funcționează rapid și nu are nevoie de date etichetate. El detectează anomaliile prin izolarea punctelor de date care sunt diferite de restul, folosind o abordare simplă și eficientă. Spre deosebire de alte metode mai complicate, precum rețelele neuronale sau modelele supravegheate, Isolation Forest are o complexitate redusă și poate fi utilizat și în sisteme cu resurse limitate, făcându-l ideal pentru monitorizarea în timp real a traficului din rețea.

Predicția traficului

Predicția traficului reprezintă procesul de estimare a comportamentului viitor al traficului din rețea, cum ar fi utilizarea lățimii de bandă sau numărul de pachete, pe baza măsurătorilor anterioare. Acest proces este important pentru planificarea rețelei, evitarea congestiilor și detectarea timpurie a problemelor. Modelele tradiționale precum ARIMA sau SVR au fost folosite în trecut, dar oferă rezultate bune doar în cazul datelor simple sau pe termen scurt.

În ultimii ani, modelele de tip LSTM au devenit tot mai populare, deoarece pot învăța automat din date de tip serie temporală și rețin atât tendințele pe termen lung, cât și fluctuațiile scurte.

Conform [Zha22], LSTM oferă o acuratețe mult mai bună față de metodele clasice și este mai potrivit pentru rețele complexe. Capacitatea acestor modele de a se adapta și de a fi scalabile le face ideale pentru predicții în timp real în rețele moderne.

Modelul Long Short-Term Memory

LSTM (Long Short-Term Memory) este un tip special de rețea neuronală recurentă creată pentru a lucra cu date secvențiale, cum ar fi serii temporale. Spre deosebire de RNN-urile clasice, LSTM reușește să rețină informații importante din trecut pe perioade mai lungi, folosind o „memorie” internă numită stare de celulă. Această memorie este gestionată de trei porți: poarta de uitare (Forget Gate), poarta de intrare (Input Gate) și poarta de ieșire (Output Gate), care controlează ce informații sunt păstrate, actualizate sau transmise mai departe. Fiecare poartă folosește funcții matematice (sigmoid și tanh) pentru a decide aceste acțiuni.

Așa cum este arătat în Figura R1, LSTM primește la fiecare pas trei elemente: noua valoare de intrare, starea anterioară ascunsă și starea anterioară a memoriei. Poarta de uitare decide ce parte din vechea memorie se elimină, iar poarta de intrare adaugă informații noi, cu ajutorul unui strat care generează valori candidate. Starea de memorie este apoi actualizată combinând vechea memorie păstrată cu noile informații, iar poarta de ieșire stabilește ce informație se transmite mai departe, formând noua stare ascunsă care este folosită în pasul următor.

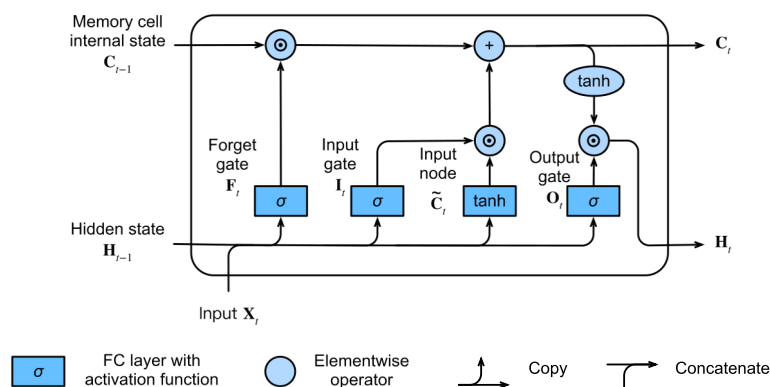


Figura R1. Arhitectura unei unități LSTM

Adaptat din Dive into Deep Learning [D2L23], licensed under Apache 2.0.

Copyright © 2023 Cambridge University Press

Modelul Isolation Forest

Isolation Forest este un algoritm de învățare automată folosit pentru detectarea anomaliilor, care, în loc să învețe cum arată comportamentul normal, încearcă să izoleze punctele din setul de date. Acesta se bazează pe ideea că datele anormale sunt rare și diferite, deci pot fi separate mai ușor.

Procesul începe prin împărțiri aleatorii ale datelor, construind un arbore de izolare (iTree) și la fiecare pas, se alege un atribut și se face o secționare aleatoare în acel atribut. Acest proces se repetă până când fiecare punct este complet izolat. Figura R2 ilustrează vizual această idee: fiecare nod al arborelui reprezintă o împărțire, iar frunzele (capetele) sunt punctele de date izolate. Se poate observa că punctele anormale ajung în frunze mult mai repede (la adâncimi mici), deoarece sunt diferite și se separă rapid. Algoritmul construiește mai mulți astfel de arbori (o pădure), iar pentru fiecare punct se calculează media adâncimilor la care a fost izolat. Acest rezultat este

transformat într-un scor de anomalie între 0 și 1. Dacă un punct este izolat în puțini pași (scor mare), este considerat anormal. Această metodă nu are nevoie de date etichetate.

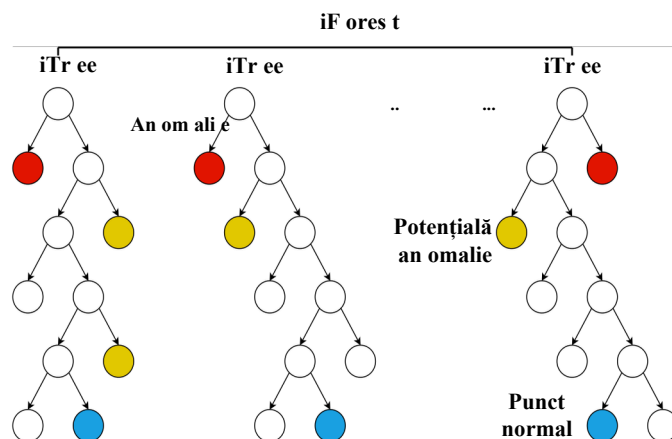


Figura R2. Arhitectura modelului IF

GNS3

GNS3 este un program folosit pentru a crea și testa rețele virtuale, fără a avea nevoie de echipamente fizice precum routere, switchuri sau alte dispozitive. Programul este împărțit în două secțiuni: o interfață grafică unde se construiește rețeaua prin „tragere și plasare” a dispozitivelor și un server care rulează sistemele de operare și asigură funcționarea acestora. Acesta poate folosi imagini reale de firmware pentru echipamente de rețea, oferind o aproximare a echipamentelor reale. Emulatorul permite și folosirea containerelor Docker, care pot simula calculatoare sau servere. Acesta este folosit și în educație de studenți care se pregătesc pentru certificări și în cercetare pentru testarea de noi rețele sau protocoale.

Interfața grafică este împărțită în două secțiuni: în prima secțiune se află echipamentele de rețea disponibile, iar în zona centrală se află spațiul de lucru, unde se conectează dispozitivele pentru a forma topologia.

Containere Docker

Docker este folosit pentru a rula aplicații în medii izolate, numite containere. Acestea includ aplicația împreună cu toate dependențele necesare, asigurând funcționarea identică pe orice sistem care are Docker instalat, independent de sistemul de operare. Spre deosebire de mașinile virtuale, containerele sunt mai rapide și folosesc mai puține resurse, deoarece împart nucleul sistemului gazdă. Figura R3 ilustrează arhitectura Docker: sistemul de operare gazdă oferă resursele, Docker Engine administrează containerele, iar comunicarea dintre client și daemon se face printr-un API.

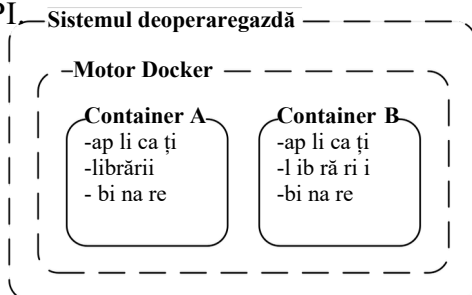


Figura R3. Arhitectura containerelor Docker [Tur24]

Fiecare container rulează izolat, cu propriul cod și biblioteci, dar toate funcționează pe același nucleu și se integrează foarte ușor în GNS3.

Unelte de monitorizare: Prometheus, Grafana și Pushgateway

În sisteme cu mai multe servicii sau topologii de rețea simulate, monitorizarea joacă un rol esențial pentru a înțelege în orice moment cum funcționează totul. În această lucrare au fost folosite trei instrumente gratuite: Prometheus, Pushgateway și Grafana. Prometheus colectează automat date numerice de la diferite surse, le salvează cu un marcaj temporal și permite interogări avansate utilizând limbajul PromQL. Interfața sa oferă acces la rezultatele interogărilor și la starea actuală a dispozitivelor monitorizate.

Pushgateway este util pentru colectarea datelor de la procese care rulează pentru scurt timp și care nu pot fi monitorizate direct de Prometheus. Grafana este folosită pentru a vizualiza aceste date sub formă de grafice dinamice și panouri personalizabile, ușor de interpretat. Utilizatorul poate crea rapid panouri noi, în care valorile colectate sunt afișate clar, prin diagrame de tip linie, bară sau alte forme vizuale intuitive, folosind interogări PromQL direct din interfață.

SNMP și Node Exporter pentru extragerea metricilor

Prometheus folosește anumite servicii care rulează pe dispozitivele monitorizate pentru a colecta metrice relevante despre ele. Două servicii gratuite și comune sunt SNMP și Node Exporter, care pun la dispoziție datele folosind cereri HTTP.

SNMP este un protocol care folosește identificatori OID pentru expunerea metricilor, iar Node Exporter este un serviciu utilizat pe dispozitive care folosesc un sistem de operare bazat pe Linux, fiecare rulând în fundal pe un port diferit (9116 pentru SNMP și 9100 pentru Node Exporter).

Figura R4 de mai jos arată parcursul metricilor în sistem de la dispozitiv până la monitorizarea lor în Grafana.

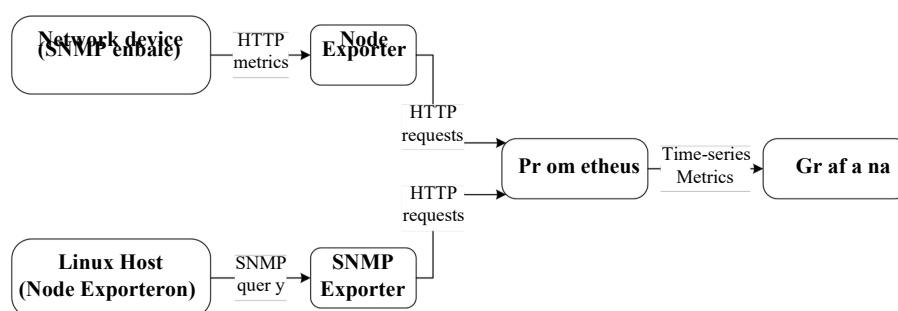


Figura R4. Colectarea metricilor în Prometheus prin SNMP și Node Exporters

3. Implementare

Instalarea GNS3 și a containerelor Docker

Emulatorul poate fi instalat de pe site-ul oficial și suportă principalele sisteme de operare (Windows 11) [GNS25]. În același mod, se instalează și GNS3 VM care rulează în interiorul

mașinii virtuale VMware Workstation Pro. Această mașină virtuală permite rularea imaginilor de routere CISCO și a containerelor Docker pentru că are Docker Engine preinstalat.

După instalarea utilităților necesare, se adaugă dispozitivele în topologie și se configurează un IP static pentru fiecare container și interfața folosită de router pentru conectarea la acestea prin specificarea parametrilor: address, netmask, gateway și DNS server.

Topologia din GNS3

Reteaua dezvoltată în GNS3 este formată din: patru containere Docker, un Router Cisco și un switch. Scopul este de a genera trafic cât mai realist care poate fi monitorizat, clasificat și prezis. Cele patru containere sunt folosite pentru a genera trafic, astfel: containerul idle este un container pornit, dar nu primește sau trimite trafic; containerul ping trimite periodic pachete ICMP; containerul TCP-HTTP generează trafic TCP prin trimiterea repetată de cereri HTTP către

o adresă specifică; containerul anomaly este folosit pentru a genera trafic anomal prin trimiterea de comenzi ping la intervale random sau în rafală.

Pentru accesul la metricile de pe containere, routerul are configurat mecanismul de port forwarding, iar fiecare container rulează un serviciu Node Exporter pe un port diferit pentru acces individual la metricile fiecăruia. Iar interfața GigabitEthernet2/0 este folosită între routerul din GNS3 și mașina gazdă și este configurată în modul bridge pentru accesul din exterior la echipamentele din topologie și conexiune la Internet.

Instalarea și configurarea uneltelor software de monitorizare

Pentru a colecta și vizualiza datele din interiorul rețelei se folosesc mai multe utilitare și servicii care folosesc porturi diferite: Node Exporter rulează sub formă de serviciu și expune metricile de sistem pentru fiecare container folosind un port specific [Pro25]; SNMP este un protocol pentru routere prin care se expun metricile pentru fiecare interfață folosind coduri OID [OID25], iar extragerea se face de către un serviciu de pe mașina gazdă (portul 9116 de obicei). Prometheus este folosit pentru a extrage la anumite intervale de timp aceste metrici de la fiecare echipament prin cereri HTTP și a le face disponibile pentru vizualizare în Grafana, dar și pentru verificarea stării acestora. Grafana este unealta prin care metricile furnizate sunt vizualizate în grafice și, în plus, include și un sistem de alertare.

Integrarea modelelor de AI pentru monitorizarea rețelei

Pentru a fi integrate în sistem, fiecare model de AI este antrenat și trimite un rezultat la intervale de 60 de secunde folosind un script de tip batch care realizează acest lucru. Apoi, serviciul Pushgateway realizează transferul datelor dintre scripturi și Prometheus pentru a fi vizualizate în Grafana, în același fel cu metricile echipamentelor.

Datele de intrare pentru modelele de AI sunt preluate din Prometheus cu o funcție în Python (fetch_multiple_metrics) care construiește o cerere de tip PromQL folosind parametrii: o listă a metricilor (metrics), IP-ul dispozitivului (instance), un interval de timp definit de două variabile (start și end) și un interval de pas setat la 60 de secunde (step). După ce scriptul furnizează rezultate de predicție sau detecție de anomalii, funcția push_metrics trimite rezultatele către Pushgateway.

Scriptul de predicție a traficului (`traffic_predictor.py`) folosește date furnizate prin SNMP de la routerul din topologia GNS3 pentru a anticipa valoarea viitoare a traficului de rețea [Fon18], [GNS25]. Sunt necesare cel puțin 11 puncte de date, care sunt scalate între 0 și 1 cu `MinMaxScaler` pentru a învăța mai eficient [Lyp24], [Sal24]. Datele sunt organizate în secvențe de câte 10 valori, apoi modelul LSTM este antrenat pentru a prezice următoarea valoare. Modelul este format dintr-un strat LSTM și unul dens, fiind antrenat cu optimizatorul Adam și eroarea pătratică medie (MSE) [Cal22], [Gre17]. După antrenare, predicția este transformată înapoi la scara originală și trimisă către Prometheus, împreună cu valorile reale, MSE-ul și durata rulării.

Scriptul de detecție a anomaliilor (`anomaly_detector.py`) folosește aceleași metrici SNMP și normalizează datele cu `StandardScaler` [Xia18], [Liu23]. Modelul folosit, Isolation Forest, detectează anomaliile, marcând punctele suspecte și calculând scoruri de anomalii. Rezultatele (rata anomaliilor, scorul mediu și maxim) sunt transmise și ele către Prometheus pentru monitorizare.

4. Rezultate experimentale

În acest capitol sunt prezentate trei scenarii de test, fiecare fiind bazat pe cel anterior cu un plus de complexitate. Scopul acestei abordări este de a observa cum adăugarea treptată a uneltelor de monitorizare și AI-ului pot crește controlul și performanța rețelei.

Scenariul 1: Topologia din GNS3 fără instrumente de monitorizare și AI

Fără a fi monitorizată și fără a avea elemente de AI, topologia emulată poate doar să trimită sau să primească trafic din Internet, detaliile din spate fiind invizibile. În plus, nu există nici un istoric al datelor privind comportamentul rețelei.

Traficul este generat folosind comenzi simple din Linux, cum ar fi ping sau folosind un container `webterm` care simulează activitatea unui browser web, executate de pe un container.

Astfel, această configurație este utilă doar pentru a ne asigura de funcționarea corectă a echipamentelor din rețea.

Scenariul 2: Topologia din GNS3 cu capabilități de monitorizare

În acest scenariu, aceeași topologie este monitorizată folosind utilitățile descrise anterior: Prometheus, Grafana, SNMP și Node Exporter. Prometheus expune metricile aferente containerelor care rulează serviciul Node Exporter, dar și de pe routerul care are protocolul SNMP activat. Un exemplu relevant de rezultate expuse este fluxul de date primite de un container pe o anumită interfață. Acest rezultat arată că pentru interfața `eth0` s-au primit 25,528 de octeți și asta se poate vedea mai jos:

```
# TYPE node_network_receive_bytes_total counter
node_network_receive_bytes_total{device="eth0"} 25528
```

Grafana este utilizată pentru a vizualiza aceste metrici prin grafice și panouri ușor de urmărit. În Figura R5 din scenariul următor se poate observa un astfel de grafic. De asemenea, Prometheus stochează aceste date mai mult de 30 de zile.

Scenariul 3: Rețea monitorizată cu AI integrat

Ultimul scenariu de test este o combinație a primelor două cu adăugarea elementelor de AI. Pentru trimiterea rezultatelor scripturilor de AI către Prometheus este folosit Pushgateway.

În Figura R5 se poate observa traficul prezis de scriptul pentru predicție de trafic și cel real.



Figura R5. Vizualizarea traficului prezis și cel real în Grafana

Se poate observa că cele două linii nu se suprapun la început, fiind necesare cel puțin 10 minute de trafic pentru a putea prezice corect următoarele valori. După aproximativ 10 minute cele două linii se suprapun, deci predicțiile sunt corecte. De asemenea, valorile MSE sunt foarte ridicate în cele 10 minute de la pornirea sistemului, dar apoi scad treptat odată cu suprapunerea liniilor pentru traficul prezis și cel real. Pentru a avea un istoric, datele importante, cum ar fi: timpul, valoarea prezisă, valoarea reală, MSE și timpul de rulare pentru fiecare rezultat sunt salvate într-un fișier de tip csv.

Pentru detecția de anomalii, se folosesc aceleași metrici, scriptul rulează la fiecare 60 de secunde și se utilizează ultimele 10 minute de trafic, iar rezultatele sunt afișate în Grafana și salvate într-un fișier CSV. La fel și pentru indicatorii de performanță: rata de anomalii - numărul de puncte marcate ca și anomalii, scorul mediu de anomalii - media scorului dat tuturor punctelor de către modelul IF și scorul maxim de anomalii - scorul maxim luând în considerare toate punctele.

5. Concluzii

Principalul obiectiv al lucrării este de a construi un sistem care poate monitoriza și analiza traficul în timp real, funcționând și fără conexiune la Internet, dar cu anumite limitări. Sistemul este configurat cu elemente de AI pentru a detecta anomalii și a prezice traficul, astfel încât să poată reacționa rapid la eventuale probleme și să optimizeze performanța rețelei. Sistemul este construit integral cu instrumente gratuite, pentru a permite reconfigurarea ușoară și accesibilă a soluției. În prezent, există câteva soluții similare în piață, dar sunt fie prea scumpe, fie nu sunt complet personalizabile. În viitor, se pot identifica soluții pentru îmbunătățirea sistemului atât pe partea de rețea, cât și asupra modelelor de AI. Astfel, modelul LSTM poate fi antrenat cu mai multe date pentru predicția de trafic, iar în cazul detecției de anomalii folosind algoritmul IF se poate regla nivelul de contaminare sau se pot combina mai multe modele pentru a reduce alarmele false. Altă posibilitate ar fi de a testa sistemul cu alte modele și a realiza o comparație pentru a identifica soluția cea mai potrivită. În topologie se pot adăuga și alte tipuri de echipamente pentru a simula o plajă mai mare de trafic.

1. Work Planning

No.	Task Name	Duration	Start	Finish
1.	State of the Art	21	14.03.2025	30.03.2025
2.	Installation of Network Emulator	10	01.04.2025	10.04.2025
3.	Installation of Monitoring Tools: Grafana, Prometheus, Node Exporter and SNMP	5	11.04.2025	20.04.2025
4.	Test Topology Configuration in GNS3	21	21.04.2025	25.04.2025
5.	Monitoring System Integration in GNS3	4	26.04.2025	05.05.2025
6.	Integration of AI in Topology	2	06.05.2025	20.05.2025
7.	Testing the System in Different Conditions	9	21.05.2025	31.05.2025
8.	Writing the Diploma Thesis	30	01.06.2025	30.06.2025

2. State of the Art

In the book [Pou24a], the author provides a solid and detailed theoretical foundation on Docker container technology, making it ideal for understanding the fundamentals and the initial setup of containers. It clearly and systematically explains the core concepts behind Docker, including application isolation, resource management, and the benefits of using containers in software development. The book offers an in-depth analysis of Docker's architecture and its key components, such as images, containers, volumes, and networks, as well as standard procedures for installation and configuration. The focus is on a rigorous theoretical approach, providing a deep understanding of Docker's internal mechanisms. Since it thoroughly explains the fundamental principles required for the correct implementation of this technology, it serves as a valuable resource for mastering Docker. However, the book does not include detailed use cases or complex practical examples related to advanced containerization scenarios. Instead, it prioritizes foundational concepts and initial configurations over specific real-world applications.

Also, the author for the [Pou24a], provides a comprehensive theoretical foundation, clearly explaining core Kubernetes concepts such as pods, services, deployments, and orchestration principles in [Pou24b] book. While Kubernetes is powerful for production environments, it introduces unnecessary complexity for GNS3 network simulations. GNS3 has built-in support for Docker containers, making it an ideal choice for lightweight network topology. Unlike Kubernetes, which requires a control plane, API server, and complex networking configurations, Docker containers in GNS3 can be deployed instantly, consuming fewer system resources. Additionally, Kubernetes' orchestration features - such as self-healing, autoscaling, and service discovery - are not needed for a controlled network simulation environment where all nodes are predefined.

With the growing demand for high-quality, uninterrupted network services, service providers need to ensure the reliability and performance of their networks. One challenge is to accurately measure and predict service quality, which often involves monitoring many variables. The article [Tur19] introduces a novel approach for network service management by modeling the network topology as a Finite State Machine (FSM). The paper leverages machine learning (ML) classification algorithms to classify network states based on Key Performance Indicators (KPIs) that are directly observed from network elements. The goal is to detect service degradation and identify its root causes using only measurable network data. The authors implement the approach in a simulated network environment using GNS3, performing experiments to assess the accuracy of different ML algorithms in classifying network states. The study also highlights the key KPIs that most impact service quality, with the results showing that ensemble learning methods like Gradient Boosting can achieve up to 88% accuracy in classifying network topology state

In [Alb22], the authors review various experimentation environments for data center routing protocols, highlighting the need for scalable and realistic testing platforms. Among the tools analyzed, GNS3 is a strong candidate for experimentation and learning. GNS3 provides an intuitive graphical user interface (GUI), simplifying the design and management of network topologies, which is especially useful for controlled, small-to-medium-scale experiments. Unlike some other tools, it allows for the emulation of real network images, enabling testing with actual router software images rather than relying solely on theoretical models. Additionally, while its Docker integration is not as extensive as in other platforms, it still enables container-based networking experiments, expanding its versatility. The paper also discusses data center routing protocols and the challenges of emulating large-scale networks, reinforcing the value of GNS3 as

a flexible and interactive tool for testing various network configurations in an accessible and efficient manner.

The study [Emm23] proposes a method for integrating machine learning into SD-WAN using a GNS3-based testbed. A decision-tree model, trained on traffic features such as protocol and flow duration, is used to classify packets and adapt routing strategies in real time. The simulation leverages Python automation and demonstrates how traffic classification improves bandwidth usage, anomaly detection, and overall network responsiveness. Although focused on SD-WAN, the architecture translates well to container-based test environments. It confirms the feasibility of lightweight AI integration in virtual networks like GNS3 and supports the idea that traffic behavior can be dynamically analyzed using real-time metrics. The study emphasizes that even with limited resources, ML can significantly enhance monitoring precision, especially when supported by flexible, emulated infrastructures.

The paper [Sal24] explores the application of machine learning techniques for classifying network traffic within a software-defined networking (SDN) environment. Supervised and unsupervised models such as Decision Tree and K-Means are used to distinguish traffic types like Domain Name System (DNS), Internet Control Message Protocol (ICMP), and Hypertext Transfer Protocol (HTTP). Despite the SDN context, the data handling methods and ML model structure are highly relevant for container-based architectures. The study highlights how accurate results can be obtained with limited features and lightweight models. This aligns with the goals of the present project, where containers simulate diverse traffic types. The research supports the use of real-time classification techniques for efficient traffic monitoring in virtual environments (GNS3).

The study [Lyp24] focuses on comparing various feature extraction tools for preparing network traffic data for ML classification. Through testing different pipelines, the authors demonstrate how small changes in preprocessing significantly affect model performance. The study is especially relevant to this project, where metrics from Node Exporter and SNMP must be preprocessed before AI analysis. It highlights the critical role of choosing the right features and ensuring data consistency. Tools that generate clean and relevant features lead to more accurate predictions and reduce false positives in anomaly detection. This underlines the importance of preprocessing in building AI-based monitoring systems.

The article [Fon18] presents GNS3 as an effective simulation environment for testing and training in network design. The authors describe how GNS3 enables users to create topologies using both virtual machines and Docker containers, allowing for flexible and realistic experimentation. Lightweight containers are used to simulate traffic endpoints and integrate with monitoring tools. This mirrors the current project's approach, where containers generate traffic and expose metrics via Node Exporter. The paper validates GNS3's ability to support data collection and ML testing without requiring physical hardware. It confirms GNS3's suitability as a platform for network observability research and prototyping.

Prometheus is described in [Pro25] as a robust, open-source monitoring solution designed for scalable metric collection and time-series analysis. It uses a pull-based mechanism to scrape data from exporters like Node Exporter and SNMP Exporter, labeling metrics for real-time filtering and alerting. In this thesis, Prometheus acts as the central collector, gathering data from containers and the router. Its structure supports integration with Grafana for visualization and with AI models for analysis. The paper highlights Prometheus' efficiency and flexibility in distributed systems, making it ideal for modern, automated, and AI-ready network monitoring environments like the one developed in this thesis.

Tool	Key Features	Docker Integration	Pros	Cons	Ideal for
GNS3	Comprehensive, multi-vendor, intuitive GUI	Excellent – native support	Highly flexible; active community	Steeper learning curve	Complex simulations with containers
EVE-NG	Web-based lab, multi-vendor support	Good – supports Docker nodes	Scalable; enterprise-ready	Complex setup; hardware	Enterprise network labs
Cisco Packet Tracer	Cisco-focused simulation	Limited – no Docker support	User-friendly; great for learning	Limited to Cisco devices	Basic learning and certification training
Cisco VIRL	Official Cisco simulation	Limited	Realistic Cisco environment	Expensive; less flexible	Cisco network scenarios
Mininet	Lightweight, SDN-focused	Partial – experimental	Ideal for SDN research; quick setup	Doesn't fully mimic real devices	SDN experiments
NS3	Detailed academic simulation	Not applicable	High accuracy for theoretical studies	Complex interface; not practical	Academic research

Table 1. A comparison between the most common network emulators [Pyn25]

In their work [Xia18], the authors propose a novel approach to anomaly detection in network management systems using the Isolation Forest (iForest or IF) algorithm. They exploit the fundamental characteristic of this algorithm, which assumes that anomalous points can be more easily separated from the rest of the data. As such, anomalies typically have shorter path lengths within the constructed isolation trees, while normal data points require more repartitioning to be isolated and thus exhibit longer paths. This key property allows IF to detect outliers without relying on distance metrics, density estimation, or labeled data, making it a powerful unsupervised learning method especially suitable for high-dimensional and large-scaled datasets. The authors enhance the effectiveness of IF for time-series network data by applying statistical feature extraction techniques that transform raw metrics (e.g., traffic, delay) into forms that highlight deviations and reduce noise or periodicity. These extracted features are then used as input to the model, enabling accurate identification of traffic anomalies, latency spikes, and similar irregularities. The approach is computationally efficient, with linear time complexity, and is well-suited for integration into real-time monitoring and detection scripts, where quick response and minimal overhead are critical.

The article [Zha22] proposes a lightweight and practical method for network traffic prediction using Long Short-Term Memory (LSTM) networks. The approach takes historical traffic measurements and uses LSTM to capture temporal patterns in the data, allowing it to learn how network values evolve over time. To improve the relevance of predictions, the model incorporates a simple weighting mechanism that emphasizes the most influential parts of the input sequence. It operates on standard traffic features such as packet sizes and flow statistics, commonly available in monitoring setups. The model was tested on real network traffic data and showed better accuracy than baseline methods, as measured by common metrics like Mean Absolute Error (MAE) and RMSE Root Mean Squared Error (RMSE). Due to its simplicity and efficiency, the method is well-suited for integration into network simulation environments such as GNS3, with data collection via Prometheus, where fast and accurate short-term forecasting is needed.

3. Theoretical Fundamentals

3.1 Anomaly Detection

Anomaly detection is the process of finding unusual patterns or values in data that do not match expected behavior. These anomalies, also called outliers, often indicate problems such as failures, attacks, or errors. Because of this, anomaly detection is essential in monitoring networks, systems, or services in real-time environments, where quick response is needed. Traditional rule-based or statistical methods often fail to keep up with modern data, which is complex, high-volume, and continuously evolving. As a result, more flexible and scalable techniques are needed to detect anomalies reliably in these conditions, especially in dynamic or unpredictable networks.

One such method is the Isolation Forest (IF) algorithm, which identifies anomalies by separating data points through random partitioning. The key idea is that anomalies are rare and different, so they can be isolated more quickly than normal data. The algorithm builds multiple trees and calculates how deep each data point appears. Anomalies usually lie closer to the root, requiring fewer splits. This makes the method fast and memory-efficient, and suitable even when no labeled anomalies are available. Its performance holds up well in dynamic data environments with multiple variables and real-world variations in behavior [Xia18].

Supervised machine learning models such as Decision Trees, Random Forests, SVMs, and k-Nearest Neighbors are also used when labeled data is available. These models learn to distinguish between normal and abnormal behavior based on past examples. Their accuracy improves when irrelevant or redundant features are removed. Genetic Algorithms help by selecting optimal inputs through evolutionary processes, while Correlation-Based Selection keeps only features that strongly relate to the target output. This makes the models more efficient, especially in large and noisy datasets where complexity can affect performance [Alt24].

Deep learning models like Convolutional Neural Networks (CNN) provide another approach by automatically extracting useful features from raw data. CNNs process inputs through multiple layers that capture both simple and complex patterns, making them well suited for detecting anomalies in structured data such as images or time series. They can operate in both supervised and unsupervised scenarios, offering flexibility in various applications. CNNs are especially powerful in visual or sequential data, where manual feature extraction is difficult [Jai24].

Autoencoders are another unsupervised method. They aim to reconstruct the input data, and if the reconstruction fails, it suggests the presence of unfamiliar or abnormal data. Likewise, Generative Adversarial Networks (GAN), composed of a generator and a discriminator, identify anomalies by comparing new inputs with the learned distribution of normal data. These models are especially useful in domains where anomalies are subtle or hard to define clearly, such as medical diagnostics or manufacturing inspection, and where labeled data is difficult to obtain [Jai24].

Choosing the right method depends on the data and context. Isolation Forest, being fast and capable of handling unlabeled data, is suitable in many practical scenarios. While classical models require clean labels and deep learning needs more resources, Isolation Forest often offers a more efficient and adaptable solution. It strikes a balance between simplicity, performance, and real-time application needs, making it effective in a wide range of monitoring systems and environments.

3.2 Traffic Prediction

Traffic Prediction refers to the process of forecasting future network traffic behavior, such as bandwidth usage, packet counts, or data rates, based on past measurements. It plays a crucial role in network planning, congestion control, and anomaly detection, allowing systems to adapt proactively rather than reactively. Inputs typically include historical traffic data, time features (like hour or weekday), and sometimes external variables such as service load or weather conditions [Zha22].

A traditional approach used in time-series forecasting is the Auto Regressive Integrated Moving Average (ARIMA) model, which combines autoregression and moving averages with differencing to handle trends. While ARIMA performs well on data with linear patterns and stationary behavior, its accuracy declines in complex, noisy environments like network traffic, where non-linearities are common.

Support Vector Regression (SVR) is another method frequently applied to traffic forecasting. It can model non-linear relationships using kernel functions and has shown solid performance in short-term prediction tasks. However, it still requires careful manual feature selection and struggles with long-term dependencies [Lyp24].

In contrast, LSTM networks are designed to handle sequences with long-term patterns, making them especially suitable for traffic prediction. LSTM models can automatically learn from raw time-series data using internal memory cells and gates to capture both short-term fluctuations and long-term trends. Studies like [Zha22] have demonstrated that LSTM significantly outperforms ARIMA and SVR in terms of prediction accuracy, especially when evaluated using common metrics such as Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE).

Because of their ability to model complex temporal relationships without manually selected features, LSTM-based models have become the preferred choice in many modern traffic forecasting systems. Their scalability and robustness make them well-suited for both real-time applications and large-scale deployments [Zha22].

3.3 Long Short-Term Memory Model

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) built to handle sequential data, like time-series, speech, or text. It overcomes the vanishing gradient problem common in standard RNNs, which struggles to remember information from many steps earlier in a sequence [Gre17].

LSTM introduces a cell state - a form of memory that persists across multiple steps - and three gates that regulate information flow. These gates use the sigmoid function (which outputs between

0 and 1) and the tanh function (which outputs between -1 and 1) to decide whether to keep, update, or output information [Wik25a].

To understand how LSTM works, it is important to look at its architecture. Each unit has a specific structure made of a memory cell and three gates that control the flow of information. These elements help the network decide what to keep, what to update, and what to pass forward,

allowing it to manage long-term information better than standard RNNs [Gre17],[Cal22], [Wiki25].

As illustrated in Figure 1, the LSTM receives three inputs at each time step: the current input value x_t , the previous hidden state h_{t-1} , and the previous memory (cell) state C_{t-1} . These are processed in parallel by several small neural networks that make independent decisions based on learned parameters. Each gate transforms the input data using simple activation functions and then passes the results to the next stage of computation [Cal22].

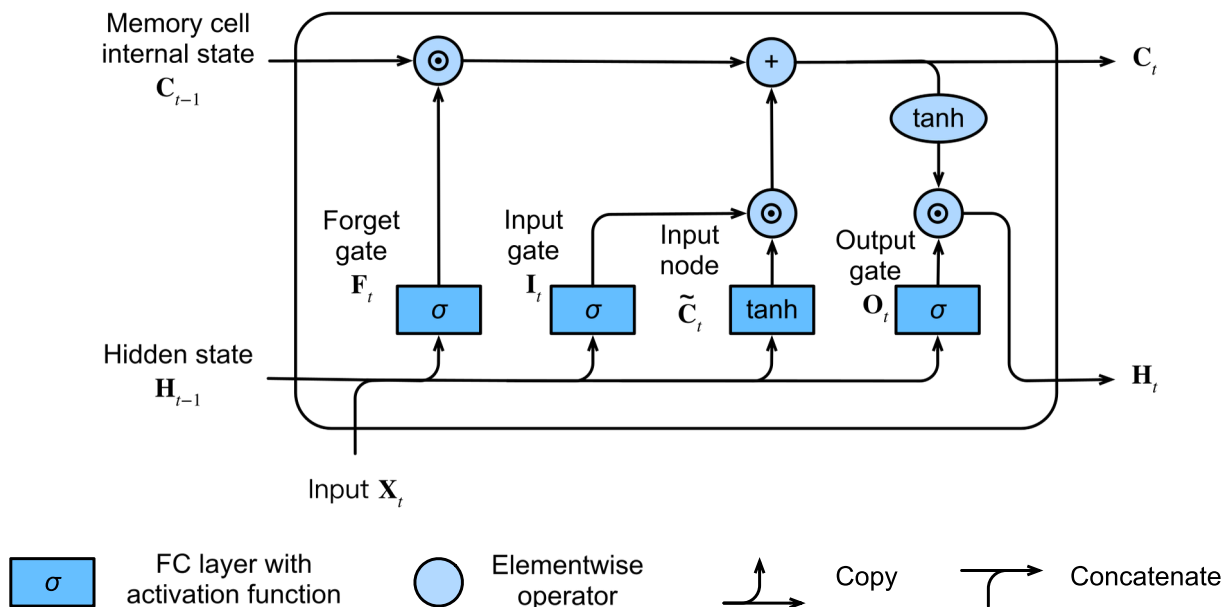


Figure 1. Architecture of an LSTM Unit

Adapted from Dive into Deep Learning [D2L23], licensed under Apache 2.0.

Copyright © 2023 Cambridge University Press

The ForgetGate (labeled in Figure 1 as ForgetGate) evaluates which parts of the previous memory C_{t-1} are no longer useful. It produces a set of values between 0 and 1, which act as a filter: values close to 1 allow the corresponding memory content to remain, while those near 0 erase it. This mechanism ensures that irrelevant or outdated information does not accumulate in memory over time, keeping the system efficient and focused on relevant context [Cal22; Gre17].

The InputGate (shown as InputGate in the diagram) is responsible for deciding what new information should be stored in the memory. At the same time, a separate component—often referred to as the candidate value generator—produces a vector of new potential content for the memory, using the tanh function to constrain the values between -1 and 1. The gate's sigmoid output determines how much of this new content is added, allowing the model to adaptively learn which inputs matter the most at each step [Cal22].

Cell State Update—the updated memory, or cell state C_t , is the result of blending two flows: the part of the old memory that was kept (after passing through the forget gate), and the new candidate values selected by the input gate. This update process gives the LSTM its "long-term memory" feature, enabling it to carry important information forward across many steps. This horizontal path across the center of the LSTM block (marked as Cell State in Figure 1) allows uninterrupted flow of important internal signals while discarding noise. The ability to retain or refresh this

memory based on changing inputs makes LSTM ideal for modeling complex, dynamic sequences [Cal22].

Finally, the Output Gate (indicated as Output Gate in Figure 1) determines what information should be sent out of the unit at the current time step. It evaluates the current input, the previous hidden state, and the freshly updated memory C_t . The memory is first passed through a tanh activation and then multiplied elementwise by the gate's sigmoid-filtered output. This product becomes a new hidden state h_t , which serves both as the visible output of the current step and part of the input for the next one. This step completes the LSTM's cycle of decision, storage, and transmission of relevant information between units [Gre17; Cal22].

Figure 1 presents a visual overview of the LSTM unit, summarizing how the components described above are connected and interact. The main elements - Forget Gate, Input Gate, Candidate Memory (tanh), Cell State, and Output Gate - are clearly labeled. The diagram highlights the flow of data through these blocks and helps illustrate the step-by-step transformation of information within the LSTM cell. Rather than introducing new mechanisms or components, it reinforces the logic behind how memory is filtered, updated, and forwarded at each time step [Cal22].

At each step, the LSTM cell performs a series of operations to process the current input and update its memory and output. These steps happen in a fixed order and follow the internal logic of the gates and memory path described above, ensuring consistency and interpretability:

1. Forget gate decision: the cell first decides which parts of the previous memory state C_{t-1} should be removed. The forget gate receives the input x_t and previous hidden state h_{t-1} , applies a sigmoid activation, and produces a vector of values between 0 and 1. Each value controls how much of the corresponding memory element is kept or forgotten, depending on the importance of past information [Cal22; Gre17].
2. Input gate and candidate generation: next, the cell prepares to update the memory. The input gate also processes x_t and h_{t-1} , and uses a sigmoid function to decide which memory units can be changed. At the same time, a separate layer creates candidate values using the tanh activation function, which maps values between -1 and 1. These two results are multiplied together to determine what new information should be added to the internal memory [Wik25a].
3. Memory state update: the cell then updates its memory state C_t by combining the two results: the filtered old memory from the forget gate and the filtered candidate values from the input gate. This gives the new cell state, which now holds both long-term context and recent updates that are useful for the task at hand [Gre17].
4. Output gate and hidden state calculation: the output gate selects what part of the updated memory to send forward. It takes the input x_t , previous hidden state h_{t-1} , and updated memory h_t , applies a sigmoid filter, and multiplies it by the tanh of the new memory. This gives the new hidden state h_t , which acts as the output of the current time step and is passed to the next one for further processing [Cal22; Wiki25].

These four steps repeat at every time step as the sequence progresses, allowing the LSTM to learn patterns across time and adjust its memory and outputs accordingly. The fixed structure and gate-based logic help the network stay stable during training and make better use of long-term dependencies and complex temporal relationships in the data [Gre17].

3.4 Isolation Forest Model

Isolation Forest is a machine learning algorithm designed to detect anomalies by isolating data points instead of profiling normal behavior. This method works well because anomalies are different and often easier to separate from the rest of the data.

The algorithm follows a few simple steps:

1. Random splitting: the algorithm starts with selecting a random feature and then choosing a random value within that feature's range to split the data. This is done recursively, creating a binary tree called an isolation tree.
2. Building the forest: multiple iTrees are built using different random subsets of data. Each tree gives a different way to isolate a point.
3. Measuring isolation: for each data point, the model calculates how many splits are needed to isolate it. This is called path length. Points that are very different from others tend to be isolated in fewer steps.
4. Scoring: the average path length over all trees is transformed into an anomaly score between 0 and 1. Shorter paths mean higher anomaly scores. A threshold is chosen to classify whether a point is normal or anomalous [Wiki25; Liu23].

This process is shown schematically in Figure 2, where a tree recursively splits features until each point is isolated. The shallow depth of anomalous points illustrates why they are easier to detect - they get separated faster due to their uniqueness. This diagram shows a simplified view of an isolation tree. Each node represents a split on a feature, and leaf nodes are isolated points. Anomalies usually end up at shallow depths, while normal points are deeper.

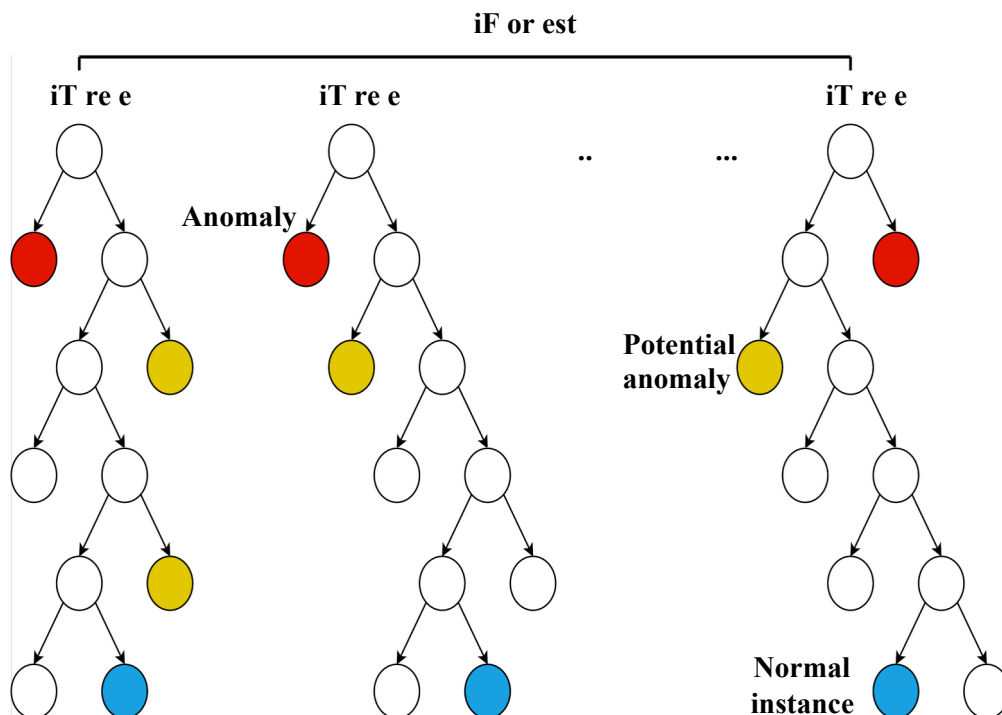


Figure 2. Isolation Forest architecture [Wen21]

To better understand how this works in practice, we can visualize how isolation behaves in two different cases. Figure 3 and Figure 4 show how a single point is isolated within a 2D Gaussian distribution. In Figure 3, the point is in a dense area of distribution. The tree needs more splits to isolate it because there are many nearby data points. This results (randomly generated) in a longer path and a lower anomaly score.

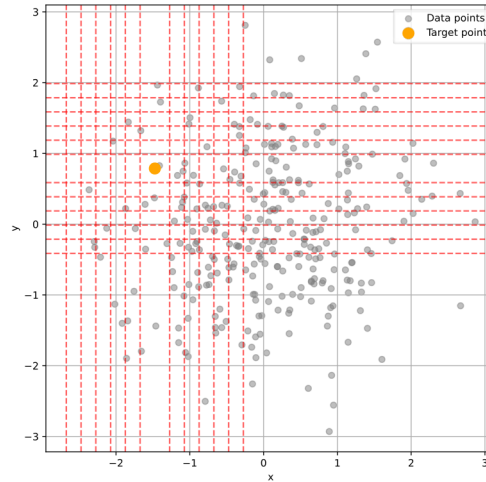


Figure 3. An example of isolating a non-anomalous point in a 2D Gaussian distribution

In contrast, Figure 4 shows a point far from the main cluster. Here, only a few splits are needed to isolate it. Because the point is different and far from others, it gets separated quickly, which results in a shorter path and a higher anomaly score.

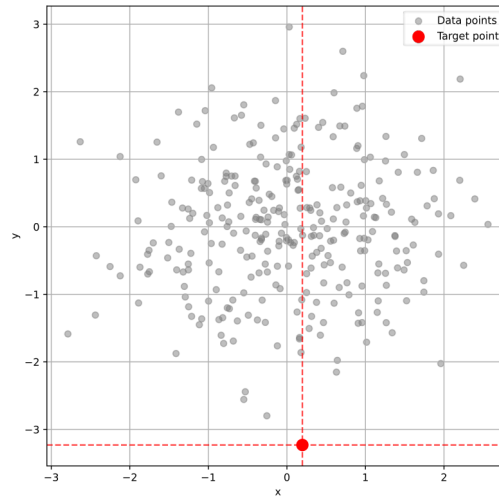


Figure 4. An example of isolating an anomalous point in a 2D Gaussian distribution

This visual intuition supports the logic behind Isolation Forest: the fewer steps it takes to isolate a point, the more likely that point is an anomaly. This makes the method very efficient, scalable, and easy to apply even when labeled anomalies are not available [Xiao18; Liu23; Wiki25].

3.5 GNS3

In the field of networking, the ability to test and configure network topologies as if they were physical systems, is a major advantage. It helps reduce costs and makes learning or testing much easier and more flexible. Instead of requiring physical routers, switches, and other equipment, nearly everything can be simulated on a computer. This approach is especially useful for students, engineers, and researchers who aim to build and experiment with networks without relying on a physical lab [Fon18].

GNS3 (Graphical Network Simulator 3) represents one of the most powerful tools available for this purpose. It enables the creation of virtual network topologies that behave very similarly to real-world networks. This makes the tool ideal for training, practicing configurations, testing new designs, and simulating failures or network issues [Fon18].

GNS3 is composed of two main components: a graphical user interface (GUI) and a backend server. GUI provides a visual environment where network designs are created by dragging and dropping devices like routers, switches, and computers onto a workspace, then linking them as if using physical cables. The backend server handles the execution of these devices, running their operating systems (OS) and managing networking tasks to ensure realistic simulation behavior.

One of the key advantages of GNS3 is its ability to emulate actual device operating systems, not just simulate their behavior. This means that real firmware images from vendors such as Cisco, Juniper, or MikroTik can be used. By running these images, users gain access to the same command-line interface and features found on real devices. This makes GNS3 far more accurate and valuable than basic simulators [Fon18].

Figure 5 shows the GNS3 interface, where users build and interact with virtual networks. A new project is created, and virtual devices such as routers, switches, firewalls, or end-user machines are added. These devices are connected by drawing links between them. Each device can be accessed through a console window, where configuration commands are entered, like working with real hardware in a lab environment.

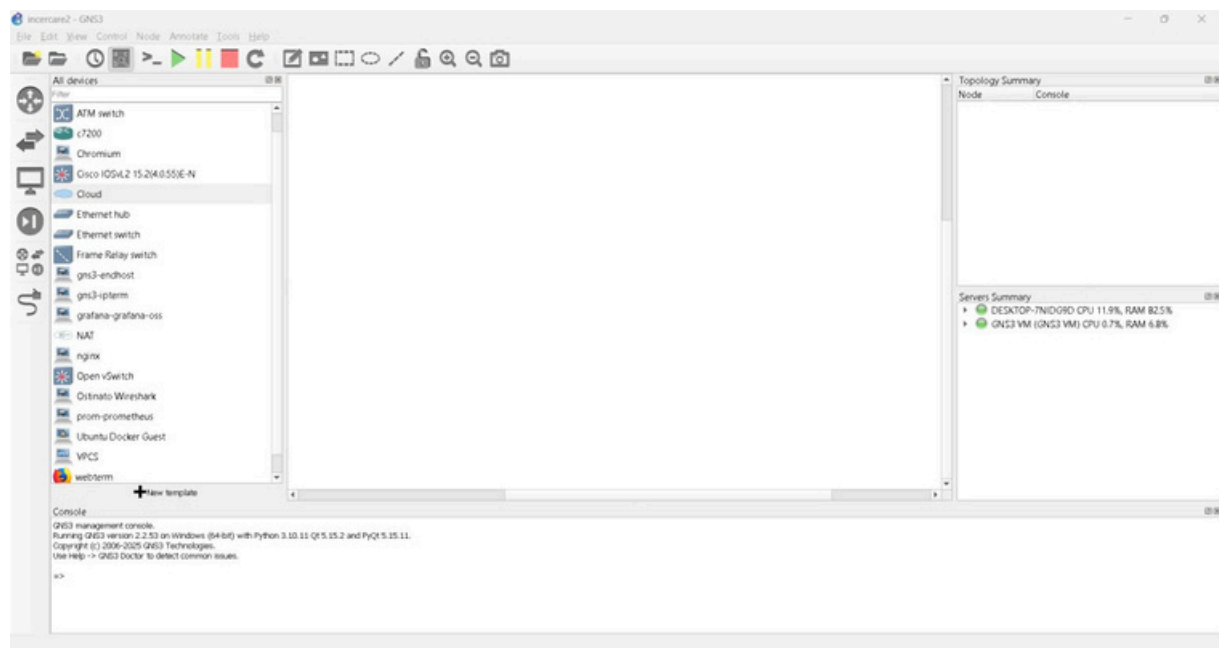


Figure 5. GNS3 GUI

GNS3 also supports the use of Docker containers, which is helpful for simulating endpoints such as client machines or servers. These containers run lightweight Linux systems and behave like devices that send or receive traffic. Specific applications, like web servers, DNS services, or monitoring tools, can also be installed and executed inside them [Pou24a].

Figure 6 shows a basic GNS3 topology, where a virtual router is connected to a Docker container acting as a server. In addition, a cloud node has been added to provide Internet access to the virtual network. This kind of setup is perfect for testing how network services behave, accessing external resources, or simulating communication between networked systems and the wider Internet.

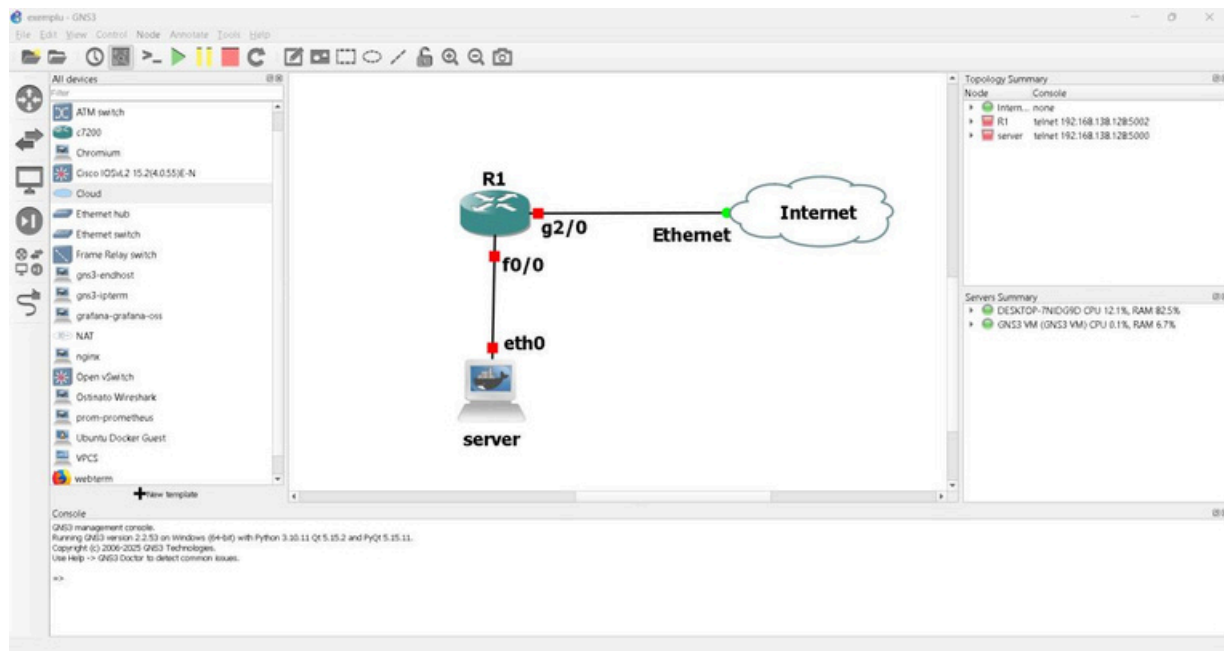


Figure 6. Simple GNS3 topology with Docker container and Cloud node [GNS25]

In education, GNS3 is commonly used by students who are preparing for certifications such as Cisco's CCNA, as it provides practical experience with real network commands and scenarios. In research, it supports the testing of new network designs or protocols before they are applied in real environments [Pyn25]. GNS3 requires a powerful computer, especially when running multiple devices or full operating system images. Although it may appear complex for beginners, the platform is supported by many tutorials and an active online community that helps users learn efficiently.

3.6 Docker Containers

In modern software development and testing environments, containerization is a widely adopted strategy for running applications in isolated and reproducible environments. Docker provides a platform that enables developers to package an application together with its dependencies into a unit called a container. These containers can operate consistently across various systems, if Docker is installed, regardless of the underlying host operating system [Pou24a].

Unlike virtual machines, which emulate full operating systems, containers share the host's kernel and are significantly lightweight in terms of resource usage. This results in faster startup times, reduced overhead, and improved scalability—making containers particularly suitable for dynamic scenarios such as continuous testing, microservice architectures, or network emulation [Pou24a].

A major advantage of Docker is its ability to ensure that applications run identically across different environments. This consistency minimizes issues related to configuration drift between development, testing, and production stages. Additionally, Docker offers tools like Docker Compose, which simplifies the orchestration of multi-container setups—useful for modeling more complex infrastructures such as simulated networks.

Figure 7 illustrates Docker’s internal architecture. At its foundation is the host operating system, which provides the system-level resources for container execution. Above this sits the Docker Engine, which acts as a central service managing all containers. It operates as a client-server system, with the Docker Daemon (dockerd) handling container lifecycles, and the Docker Client (docker) allowing users to interact with the system via command-line instructions. Communication between the client and daemon occurs through a REST API, which can also be accessed by third-party tools or scripts. Each container—such as those shown in the diagram—packages its own application code, libraries, and binaries. While they all share the same OS kernel, the containers remain isolated from one another. This design makes Docker highly efficient, quick to deploy, and capable of maintaining consistent behavior across environments [Tur24].

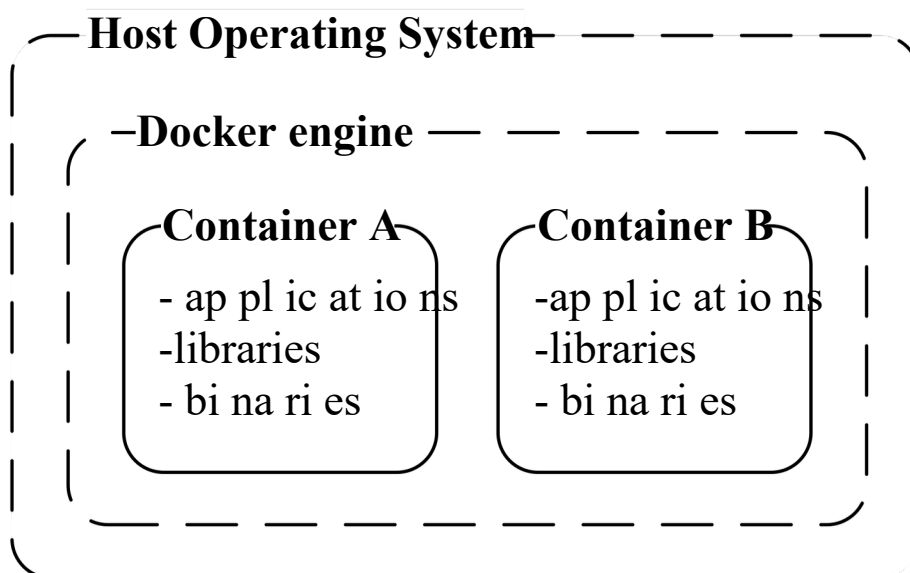


Figure 7. Docker Container architecture [Tur24]

For both academic use and real-world testing scenarios, Docker containers provide a flexible and portable foundation. They integrate seamlessly with platforms like GNS3, making it possible to launch services and monitoring tools directly within containers, while also supporting reliable automation and stable execution [Pou24a].

3.7 Monitoring Tools: Prometheus, Grafana and Pushgateway

In any system that includes multiple services, containers, or simulated network topologies, it becomes very important to monitor what is happening at every moment. Monitoring tools allow us to collect useful data about system performance, network usage, system health, and application activity. In this thesis, three popular and open-source tools were used to build a complete monitoring setup: Prometheus, Grafana, and Pushgateway. These tools work together to collect, store, and display information in a way that helps us understand the behavior of the system over time [Pro25]. This is very useful when testing and simulating network topologies

Prometheus is a standalone monitoring server designed to collect, store, and process time-series data from various systems and services. It operates as an HTTP server that runs locally or on a designated host, typically listening on a specific port—by default, port 9090. Through this interface, users can access a web-based UI, run queries, and interact with collected data [Pro25]. Prometheus scrapes metrics by making direct HTTP requests to endpoints exposed by monitored targets. Each of these targets must expose metrics at a standard path (usually /metrics), accessible over a specific IP address and port combination. The server is configured to periodically query these endpoints using HTTP GET requests, retrieving numerical data that is then stored in Prometheus's internal time-series database.

Each metric collected is associated with a timestamp and a set of descriptive labels (such as instance, job, or device) to enable precise filtering and identification. The collection process is fully automated and centralized, following a pull-based model in which Prometheus initiates data retrieval from each desired endpoint. What makes Prometheus very useful is its query language, PromQL, which allows users to extract and process metrics in real time. With PromQL, we can calculate averages, detect changes over time, compare values across devices, and much more [Pro25]. This architecture allows Prometheus to maintain complete control over the frequency, scope, and organization of the monitoring process. Its ability to address targets directly via IP and port makes it well-suited for containerized environments, simulated network topologies, or distributed systems where services may run isolated and have access to the Internet [Pou24a].

Figure 3 presents the user interface of Prometheus, showing both the query entry page and the target status page. For example, a common query might be for the metric `ifHCOutOctets`, which shows how much outgoing network traffic was transmitted through a specific interface. The result displays a list of current values for each target (which could be a container, a virtual machine, or another monitored device). Each entry includes labels that help identify where the data is coming from, such as the name of the interface or the IP address of the instance. The tool also supports filtering and mathematical operations to allow more advanced analysis. This is helpful for detecting unusual patterns, slowdowns, or failures in the system. In addition to the query interface, the second panel displays the target status view, where users can see which endpoints are currently reachable, the health of each target, and when data was last successfully scraped. This allows for real-time infrastructure monitoring and quick identification of connectivity or availability issues. Together, these interface elements provide both high-level visibility and low-level control over the monitored environment.

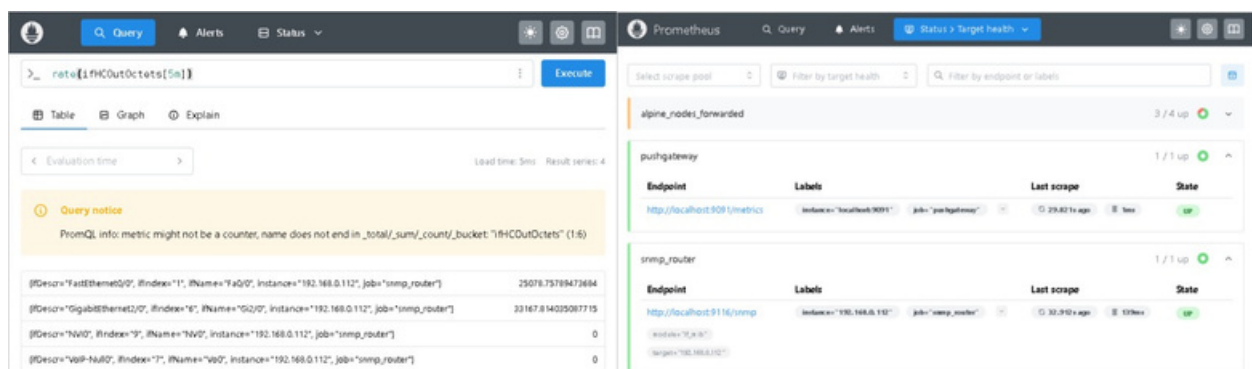


Figure 8. Prometheus Interface: Query results and target status overview [Pro25]

Pushgateway - some processes or services do not run all the time. For example, a script might run once to perform a task and then stop. These short-lived processes are not easy to monitor with

Prometheus, because Prometheus collects data only when the process is active. This is where Pushgateway becomes helpful. Pushgateway allows these processes to send their data directly to

a place where Prometheus can later pick it up. It acts like a temporary storage space for metrics. In this thesis, Pushgateway is used for collecting information from scripts or short-running tasks that are part of network testing. These scripts can push metrics such as packet delay or number of connections directly to Pushgateway. Prometheus will then access these values when it performs its regular data collection. This method ensures that even fast processes that finish quickly can still be included in the monitoring system [Pro25].

Grafana - while Prometheus is very good at collecting and storing data, it is not designed to present that data in a visual or user-friendly way. For this reason, we use Grafana, a powerful visualization tool that connects to Prometheus and displays the collected data in the form of graphs, tables, and interactive dashboards. Grafana queries Prometheus via its HTTP API, using Prometheus Query Language (PromQL) to extract specific metrics, such as network traffic, system load, memory usage, or error rates [Gra25]. These queries can be dynamically linked to visual panels that update in real time, providing a clear view of how the monitored system behaves. Grafana allows the user to build custom visual layouts, where each panel can display a different metric, aggregation, or transformation of the data. This enables the identification of patterns, bottlenecks, or anomalies that would be difficult to spot in raw numerical format. Moreover, Grafana supports alerting rules, thresholds, and filtering based on labels such as instance name, job, or interface, which adds a layer of intelligence to the monitoring setup. Grafana is typically deployed as a local or remote web application, accessible through a browser interface (e.g., at <http://localhost:3000>). It works by querying a connected Prometheus server, which collects time-series data from sources such as SNMP exporters or node exporters at regular intervals, often every 15 or 60 seconds. This setup allows Grafana to present up-to-date metrics in real time, offering users a clear and customizable way to monitor system performance, network activity, or infrastructure health through interactive dashboards [Gra25].

Figure 9 shows the Grafana interface with a newly added panel, before any metric or data source is selected. It represents the initial step in creating a custom graph for monitoring purposes.

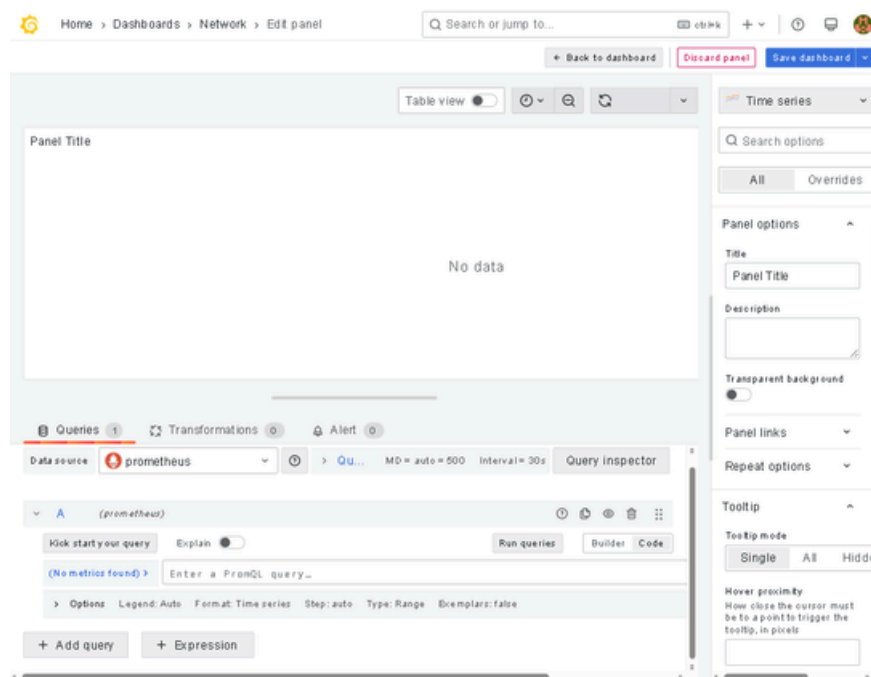


Figure 9. Grafana interface

3.8 SNMP and Node Exporter for Scrape Metrics

Prometheus relies on exporters to collect metrics from external systems. Two common types of exporters used in monitoring infrastructure components are SNMP Exporter and Node Exporter. Both act as bridges between Prometheus and the devices or systems being monitored, exposing structured metrics in a format that Prometheus can scrape over HTTP.

SNMP Exporter - the SNMP Exporter is designed to retrieve data from network devices that support the Simple Network Management Protocol (SNMP), such as routers, switches, and firewalls. SNMP works over User Datagram Protocol (UDP) and uses structured identifiers (OIDs) to expose data like interface traffic, error rates, and device status. An OID is a numerical identifier that uniquely references a specific metric or variable exposed by a device [OID25]. These identifiers follow a tree-like structure defined in standardized Management Information Bases (MIBs). For example, the OID .1.3.6.1.2.1.2.2.1.10 refers to ifInOctets, which represents the number of bytes received on a particular interface. Similarly, .1.3.6.1.2.1.2.2.1.16 refers to ifOutOctets, for outgoing bytes. The SNMP Exporter uses a configuration file (.yml file) to map human-readable metric names to these OIDs.

When Prometheus initiates a scrape, it sends an HTTP request to the SNMP Exporter. The exporter then performs an SNMP query to the device (usually on port 161/UDP), retrieves the relevant OID values, and converts them into Prometheus-compatible metrics [Pro25]. This allows Prometheus to monitor network devices in the same way it monitors systems that provide data over HTTP.

Node Exporter - is used to monitor Linux system-level metrics such as CPU usage, memory, disk I/O, network statistics, and file system activity. It runs as a background service directly on the machine being monitored and exposes metrics at an HTTP endpoint, usually on port 9100 [Pro25]. When Prometheus scrapes the Node Exporter, it retrieves hundreds of metrics formatted as plain

text. These include `node_memory_bytes_total`, `node_memory_seconds_total`, `node_network_receive_bytes_total` and many others like these. This data provides deep insight into the operating system's resource usage and performance, making Node Exporter ideal for monitoring servers and containers.

Figure 10 shows how Prometheus collects monitoring data using exporters. A network device, such as a router or switch, sends data through the SNMP protocol to the SNMP Exporter. At the same time, a Linux host (any system running a Linux-based operating system) runs the Node Exporter, which exposes system-level metrics over HTTP. Prometheus sends periodic scrape requests to both exporters, collecting the available metrics in a format it can store and process. These metrics are saved in Prometheus's time-series database and can be visualized through Grafana dashboards [Pro25].

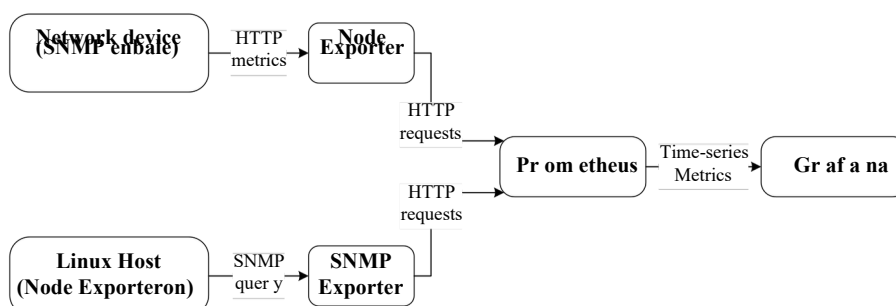


Figure 10. Prometheus metric collection via SNMP and Node Exporters

4. Implementation

This chapter presents how AI is used to predict traffic and detect anomalies in a basic network topology implemented in GNS3 using Docker containers as endpoints to generate traffic. A preliminary design was presented in [Cir25] at SSET 2025. To be more precise, the topology includes four Docker containers and each one produces a different type of traffic: Idle container does not receive or transmit traffic, a Ping container sends ICMP request at fixed intervals, a TCP/HTTP container that uses a browser to generates TCP and HTTP traffic and an Anomaly container design to simulate anomalous traffic, such as a ping flood. All these containers are connected to a router through a switch, forming the main components of the network topology. The router is further connected to the host PC via a Cloud node to provide Internet connectivity and access to each container individually for data collection and monitoring purposes. Figure X offers a visualization of this topology from GNS3, illustrating the relationships between the network components. Traffic prediction is performed using an LSTM model, which is well-suited for capturing temporal patterns, while anomalies are detected using an Isolation Forest model, known for its efficiency in identifying outliers in multidimensional data.

To collect network metrics from the GNS3 topology, Prometheus gathers data from monitoring services running on the containers and the router. This data is stored and made available for visualization, with Grafana regularly requesting metrics from Prometheus to display them as graphs and dashboards. Additionally, AI scripts analyze the collected data to provide predictions and detect anomalies, sending their results back to Prometheus 3.3.0 for integrated visualization in Grafana 12.0.

4.1 GNS3 Installation with Docker Containers

The GNS3 installer is available on the official website and supports all major operating systems, including Windows, macOS, and Linux [GNS25]. In this thesis, this emulator was installed on a Windows 11 system because it offered good compatibility with both the client and the virtualization tools needed.

During the installation process, it was important to check the option to install the GNS3 VM. This is a lightweight Virtual Machine (VM), based on Ubuntu 20.04.6.LTS Linux Operating System, that was specially created to work with GNS3. The purpose of the VM is to improve performance and simplify the use of advanced features such as Docker containers, QEMU virtual machines, and IOU images. It also helps reduce the processing load on the host computer by running network devices inside the virtual machine.

The GNS3 VM runs inside a virtualization platform. In this project, VMware® Workstation 17.6.3 Pro was used, but alternatives like Oracle VirtualBox are also supported. VMware must be installed and properly configured before the GNS3 VM can run. Once launched, GNS3 automatically connects to the VM, allowing it to communicate and share resources with the host system.

One of the main advantages of using the GNS3 VM is that it comes with several preinstalled tools, including Docker Engine. Docker Engine is a platform that allows users to download, run, and manage containers. In GNS3, these containers act as simple network devices for traffic generation.

After setting up the GNS3 environment and confirming that the GNS3 VM is running correctly, the next step is to add Docker containers. GNS3 provides an easy way to do this: users can either download preconfigured containers from Docker Hub (a public repository with thousands of images) or build their own. These containers are downloaded through the Docker Engine inside the GNS3 VM and can be reused later without needing to redownload them.

Docker containers can be managed and configured from the Docker Containers section in the GNS3 Preferences tab. Once added, these containers appear in the device list and can be used inside the network topology just like any other router, switch, or virtual machine. Each container acts as a network device and can exchange traffic with other components, such as routers or monitoring tools, inside the simulated environment.

To ensure that the containers can communicate correctly inside the network topology, it is necessary to configure a static IP address for each one. This is done by modifying the network configuration file located at `/etc/network/interfaces` inside each container. The configuration defines the IP address, subnet mask, default gateway, and DNS settings for the `eth0`

interface. A static IP ensures that the address remains the same, even after restarts, and it is valid only within the GNS3 virtual network. The configuration used in each container is the same in structure, with only the address field changing from one container to another to avoid IP conflicts. This setting ensures reliable communication between devices.

Example of a static IP configuration for a configuration file:

```
# Static config for eth0
auto eth0
iface eth0 inet static
address 10.1.1.3
netmask 255.255.255.0
gateway 10.1.1.1
up echo nameserver 8.8.8.8 > /etc/resolv.conf
```

In this example:

- address - defines the container's static IP address. This means the container always uses the same IP available only inside the topology. It is also used by monitoring tools to know which container's metrics to access using router's port forwarding features
- netmask - sets the size of the network. It tells the container which IP addresses are part of its local network. For example, with 255.255.255.0, the container sees all devices with IPs starting with 10.1.1 as being in the same subnet.
- gateway - indicates the IP address of the router's interface that connects to the containers. This is the next hop to access the Internet or to send metrics to monitoring tools. The container sends all traffic to this gateway.
- The last line sets the DNS server (Google's 8.8.8.8), allowing the container to resolve domain names when establishing Internet connections. Without this, containers can only communicate using IP addresses.

This configuration ensures that the container always has the same IP address and can communicate with other devices in the network through the correct gateway and Domain Name System server. The configuration is applied in the same way on all containers, changing only the IP address to avoid conflicts.

This setup is used because the router in the GNS3 topology has port forwarding enabled. With this feature, each container can be accessed individually from outside the virtual network using the router's external IP and a specific forwarded port. The static IP shows which container is behind each port, making the connection easier.

This is a key configuration for tools like Prometheus and Grafana, which need to collect metrics from each container in real time. These tools rely on the port forwarding set up on the router to reach the static IP addresses of the containers. With fixed IPs, Prometheus can scrape metrics from each container by sending HTTP requests to the router's external IP (obtained with DHCP), using a unique port for each container. The router redirects the request to the correct internal static IP, allowing Prometheus to collect the data accurately, then Grafana displays this data.

4.2 GNS3 Network Topology

One of the main challenges of this project is to build a network topology capable of generating realistic traffic that reflects real-world scenario, while also allowing the traffic to be monitored, classified as normal or anomalous, and predicted. To address this, the GNS3 emulator is a powerful network tool that allows us to simulate both physical network devices and Docker containers.

The network topology consists of:

- Four Docker containers for generating different types of traffic
- A Cisco Router to enable communication between monitoring tools and devices
- A switch for connection between containers and router

This setup is illustrated in Figure 11.

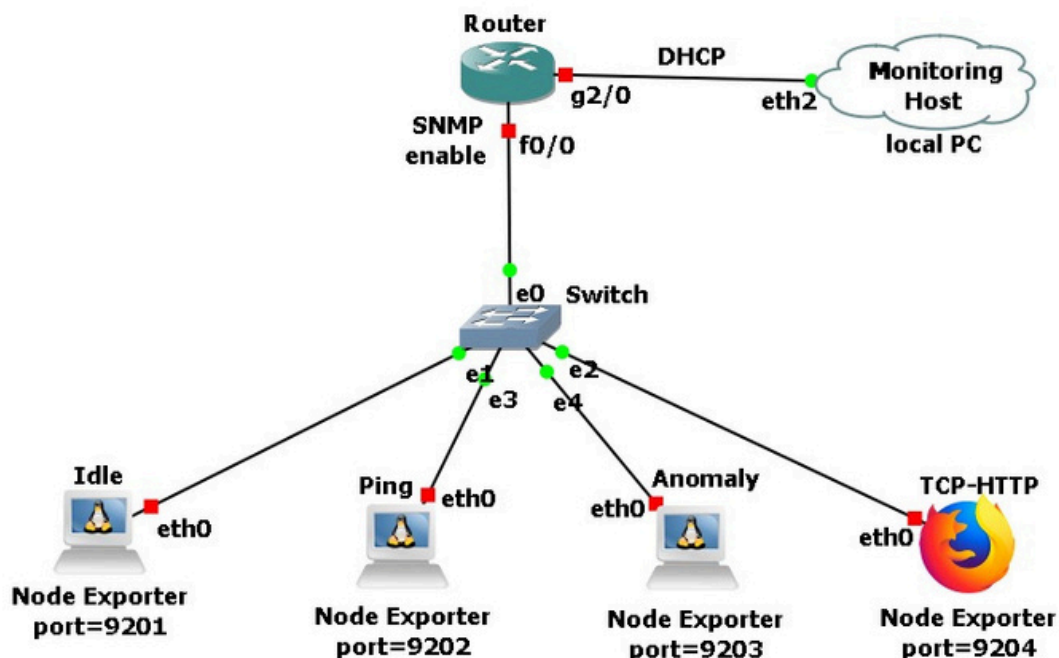


Figure 11. GNS3 network topology used for traffic monitoring and anomaly detection

Each Docker container in the network topology was selected with a specific purpose, helping to simulate different types of traffic and behaviors. These containers allow for testing how the network reacts to both normal and unusual situations. Below is a detailed explanation of each container:

- **Idle container** - this container represents a device that is powered on, but it does not send or receive data. It behaves like a desktop computer or server that is running in the background, without user activity. It is important because many real systems do not constantly generate traffic. The container allows comparison between an idle state and more active or suspicious behaviors.
- **Ping container** - this container regularly sends ICMP echo requests, which are also known as "ping" packets. It mimics a device or script that checks if other systems are reachable on the network. These pings are sent at fixed intervals and help simulate simple connectivity tests. It is useful for scenarios where devices need to check the status of other devices, like in automated monitoring or alerting systems.
- **TCP-HTTP container** - this container generates standard TCP traffic by sending repeated HTTP requests to a specified address. It simulates a user browsing web pages or a background process communicating with a web server. This type of activity is very common in real networks and helps create a realistic traffic pattern. The container shows how regular web traffic looks.
- **Anomaly container** - this container is designed to produce unusual or irregular traffic, simulating possible network problems or malicious behavior. It primarily uses the ping command, but unlike the normal Ping container, it sends packets at random intervals or in short bursts. This creates an abnormal pattern that stands out from the rest. Additionally, tools like curl can be used inside the container to send strange or repetitive HTTP requests, such as accessing rarely used endpoints or sending malformed data.

All four containers run Node Exporter that provides system-level metrics such as CPU usage, memory consumption, and network activity for individual endpoints like containers.

Accessing the metrics of each container is handled through the Cisco router, which plays a central role in connecting the internal GNS3 topology to the host machine. The router has SNMP enabled, allowing it to expose performance and traffic metrics for each of its interfaces. To enable SNMP, the following command is configured on the router: `snmp-server community public RO`.

In addition, the router is configured with port forwarding rules, like Network Adresse Translation (NAT), that redirect traffic from specific ports on its external interface to the corresponding Node Exporter ports inside each Docker container. For each container, a similar command is applied, changing only the destination IP and port to match the container's settings. For example, the command below redirects external traffic from port 9201 on the router (with IP address 192.168.0.112) to port 9201 on a container with IP 10.1.1.2. Similary commands are used for other containers, but their IP and ports are changed.

```
Router(config)#ip nat inside source static tcp 10.1.1.2 9201 192.168.0.112 9201
```

This setup makes it possible for monitoring tools (Prometheus and Grafana) to access the metrics of each container individually by connecting to the router's external IP and the forwarded port.

To establish communication between monitoring tools and the network topology, a Cloud node is used in GNS3. This node is linked to a network interface of the GNS3 VM, which runs on VMware Workstation Pro. The interface is set to bridge mode, meaning it works like a physical

network adapter connected to the same local network as the host PC. As a result, the router inside GNS3 obtains its IP address automatically through Dynamic Host Configuration Protocol (DHCP), provided by the physical router on the same network. To enable this, the following commands are configured on the GNS3 router's interface connected (GigabitEthernet2/0) to the Cloud:

```
Router(config)#  
  
        interface GigabitEthernet2/0  
Router(config-if)#ip address dhcp  
Router(config-if)#ip nat outside  
Router(config-if)#no shutdown
```

The interface GigabitEthernet2/0 command selects the interface on the. In this setup, the GigabitEthernet2/0 interface is used to create a link between the virtual router from GNS3 and the Cloud node. The Cloud node acts as a bridge that connects the GNS3 topology to the real physical network - a router in this thesis. This connection is important because it allows the Docker containers inside GNS3 to have Internet access and be accessible from outside the network for metric collection.

The ip address dhcp command tells the router to automatically request an IP address from the physical router, using the DHCP protocol. This avoids manual IP configuration and ensures that the router can communicate with the host machine and the Internet.

The ip nat outside command is used to define this interface as the external or "outside" side of the NAT configuration. This is important because the router needs to know which interface handles traffic going or coming from outside the GNS3 virtual network. NAT allows devices inside the GNS3 topology, like Docker containers, to be accessed from outside using external IP and specific ports.

Lastly, the no shutdown command is used to activate the interface. By default, router interfaces are turned off (administratively down), so this command is necessary to set the interface up and allow it to send and receive traffic.

Together, these commands prepare the interface for external communication, enabling both Internet access for containers and visibility from tools like Prometheus and Grafana that run outside the GNS3 environment. So, we can access each individual container and its metrics through the router's IP address combined with the specific port where the Node Exporter is running. Similarly, the router's interface metrics (exposed by SNMP) can be accessed directly via the router's IP address.

The following commands are used to configure the FastEthernet0/0 interface on the router. This interface connects directly to the Docker containers inside the GNS3 network. It acts as the gateway for the containers, meaning its IP address is used to send traffic outside GNS3 network. Below are the commands:

```
interface FastEthernet0/0  
ip address 10.1.1.1 255.255.255.0  
ip nat inside
```

The command selects the interface that links the router to the internal Docker network. The command sets a static IP address and subnet mask for this interface. The IP 10.1.1.1 is important because it is used as the gateway address in every container's network configuration file. This means that all traffic sent

by the containers goes to this address first, being the first hop (the first step in sending data to monitoring tools or the Internet).

The `ip nat inside` command marks this interface as the internal side of the NAT setup. This command pairs with the `ip nat outside` command from the `GigabitEthernet2/0` interface.

These settings allow traffic to move between the containers and the outside of GNS3 topology through the router, enabling tools like Prometheus and Grafana to collect data using port forwarding.

4.3 Installation and Configuration of Monitoring Tools

To collect, store, and visualize the metrics generated within the network topology, a set of monitoring tools was used. These tools work together to provide real-time insights into both system performance and network activity, as well as to display results from AI scripts that detect anomalies and predict traffic. Three tools were used for this purpose: Grafana for visualizations, Prometheus and Pushgateway for scraping. All these are running on local host, each on a configurable port, but usually: 3000 - Grafana, 9090 - Prometheus and 9091 - Pushgateway.

This setup ensures that the tools are fully accessible without requiring an Internet connection. All of them are open-source and freely available for download from their official websites [Gra25; Pro25].

As the main data collector, Prometheus manages all incoming data before being visualized or analyzed. It works by periodically sending HTTP requests (called scrapes) to endpoints that expose metrics in a specific format. In this thesis, Prometheus collects data from two types of endpoints: Docker containers (via Node Exporter) and Cisco Router (via SNMP). This integration is achieved by specifying the endpoints (targets) in the Prometheus configuration file (`prometheus.yml`). Below is a snippet of the Prometheus configuration file used, written in YAML format. It defines how Prometheus collects metrics from Docker containers, the Pushgateway, and the Cisco router in the GNS3 topology:

```
global:
  scrape_interval: 15s      #how often to scrape metrics
scrape_configs:
  - job_name: 'alpine_nodes_forwarded'
    static_configs:
      - targets:
          - '192.168.0.112:9201' #idle container's node exporter port
        labels:
          container: 'alpine-idle' #label for this container
      - targets:
          - '192.168.0.112:9202' #ping container
        labels:
          container: 'alpine-ping'
      - targets:
          - '192.168.0.112:9203' #anomaly container
        labels:
          container: 'alpine-anomaly'
      - targets:
          - '192.168.0.112:9204' #tcp-http container
        labels:
          container: 'alpine-http-tcp'
  - job_name: 'pushgateway'
```



```

static_configs:
  - targets: ['localhost:9091'] #pull metrics from local pushgateway

- job_name: 'snmp_router'
  static_configs:
    - targets: ['192.168.0.112'] #gns3 router ip address
  metrics_path: /snmp #use snmp metrics path
  params:
    module: [if_mib] #snmp module for network interfaces
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target
    - source_labels: [__param_target]
      target_label: instance
    - target_label: __address__
      replacement: localhost:9116 #snmp exporter runs locally on this port

```

This configuration file defines how Prometheus collects and processes data. At the top, under the global section, the settings specify that data are scraped every 15 seconds (`scrape_interval: 15s`). This interval is set to control how frequently Prometheus updates its data.

The first job, named `alpine_nodes_forwarded`, collects system metrics from Docker containers running inside the GNS3 network. Prometheus accesses these containers through the router's IP address (192.168.0.112) using different ports (9201-9204). Each container runs its own Node Exporter service, and this service listens to a unique port, that collects system-level metrics such as CPU usage, memory usage, disk space, and network activity and then exposes it. These metrics help monitor the health and performance of each container. Besides this, Prometheus exposes many other metrics from both router and containers, including CPU usage, memory usage, the amount of incoming traffic, and the status of each network interface.

Node Exporter runs inside each container as a background process. Once started, it automatically begins to collect system data and makes it available on a local HTTP endpoint. Prometheus connects to this endpoint using port (forwarded by the router) and router's IP and reads the metrics at regular intervals (`scrape_interval: 15s`). This setup allows each container to be monitored separately, even though they all share the same external IP address (the router's IP).

The labels in the configuration help organize the data, so Grafana can identify which metrics belong to which container ('`alpine-idle`', '`alpine-ping`', '`alpine-anomaly`', '`alpine-http-tcp`') instead of using their IP and port.

The second job, `pushgateway`, targets the local Pushgateway service running on `localhost:9091`. Pushgateway is used to collect and temporarily store data that is manually sent by AI scripts used for traffic prediction and anomaly detection. Since these scripts do not run all the time and cannot be scraped regularly like containers or routers, they push their results directly to Pushgateway every 60 seconds. Prometheus then picks up this data the next time it scrapes Pushgateway. This way, important metrics generated by AI models are not lost and can still be included in the monitoring dashboard. Pushgateway can be installed from the official Prometheus website [Pro25]

The third job, `snmp_router`, is configured to collect SNMP metrics from the Cisco router within the GNS3 topology using its IP address. It specifies the metrics path `/snmp` and the SNMP module `if_mib`, which focuses on network interface-related data such as traffic statistics and errors. The configuration also includes relabeling rules to properly route the SNMP requests through a local

SNMP Exporter running on localhost:9116, which translates SNMP data into a format Prometheus can understand. The SNMP Exporter uses a YAML configuration file that tells it exactly which metrics to collect from the Cisco router. This file contains lists of OIDs, which are unique codes representing specific data points in the device's MIB. Below is an example snippet from the SNMP Exporter configuration file that defines a metric for monitoring the incoming traffic on a network interface:

```
- name: ifInOctets

  oid: 1.3.6.1.2.1.2.2.1.10
  type: counter
  help: The total number of octets received on the interface, including framing
  characters.
  indexes:
    - labelname: ifIndex
      type: gauge
  lookups:
    - labels:
        - ifIndex
        labelname: ifAlias
        oid: 1.3.6.1.2.1.31.1.1.1.18
        type: DisplayString
    - labels:
        - ifIndex
        labelname: ifDescr
        oid: 1.3.6.1.2.1.2.2.1.2
        type: DisplayString
    - labels:
        - ifIndex
        labelname: ifName
        oid: 1.3.6.1.2.1.31.1.1.1.1
        type: DisplayString
```

This configuration defines a metric called `ifInOctets`, which counts the total number of bytes received on a network interface. OID is the SNMP identifier for this specific data. The metric is of type `counter`, meaning it only increases over time. The `indexes` section tells Prometheus how to identify which interface each metric value belongs to, using `ifIndex` as a label. The `lookups` section links the numeric interface index to human-readable labels like `ifAlias`, `ifDescr`, and `ifName`, which provide more detailed descriptions of each interface. These SNMP metrics can be accessed through a web browser at address: `http://localhost:9116`. Each metric is displayed with detailed labels for every interface on the router. For example, the `ifInOctets` metric shows how many bytes have been received on each network interface. A sample result returned by SNMP Exporter, showing the `ifInOctets` values for two interfaces (`Fa0/0` and `Gi2/0`) on the router:

```
ifInOctets{ifAlias="",ifDescr="Fa0/0",ifIndex="1",ifName="Fa0/0"} 567247
ifInOctets{ifAlias="",ifDescr="Gi2/0",ifIndex="6",ifName="Gi2/0"} 256082
```

Each line shows the number of bytes received at the end and labels like `ifDescr` and `ifName` help identify which physical port the value refers to.

This setup lets Prometheus collect all the important data from the Docker containers and the router, so the network and system can be watched in real time, and the data can be used later by AI analysis. Each metric is stored with a timestamp, which allows tracking how the system behaves over time. This is useful not only for monitoring but also for detecting problems early or understanding what happened in the past during specific events.

To visualize data collected by Prometheus, we use Grafana, which turns complex metrics into clear, interactive graphs and dashboards, making it easier to monitor and analyze network performance in real time, capabilities that Prometheus does not offer. Grafana connects directly to Prometheus as a data source and regularly queries it for updated metrics. It organizes this data into customizable dashboards with various panels like graphs, charts, and tables. Users can interact with these dashboards to select a specific time frames, apply filters, or compare different metrics. This visual approach helps quickly identify trends, spikes, or anomalies in network and system performance.

Grafana supports many types of charts and graphs, such as line graphs, bar charts, gauges, and tables. These visual elements are very customizable, so users can change colors, labels, sizes, and even the way data is grouped or filtered. This flexibility makes it easy to build dashboards that fit exactly what the user needs, whether it's for system monitoring, traffic analysis, or performance trends.

Grafana also supports alerts, which are useful for keeping track of system health without having to watch the dashboard all the time. Users can set alerts to trigger if a specific metric goes above or below a certain limit. When that happens, Grafana can send a notification by email, chat, or other tools. This helps detect problems early and take action quickly, keeping the system stable and reliable.

Figure 12 shows an example of a Grafana dashboard used in this thesis. The dashboard displays key metrics such as CPU and RAM usage, network traffic, and active processes per minute, providing a clear overview of the system's health and activity.

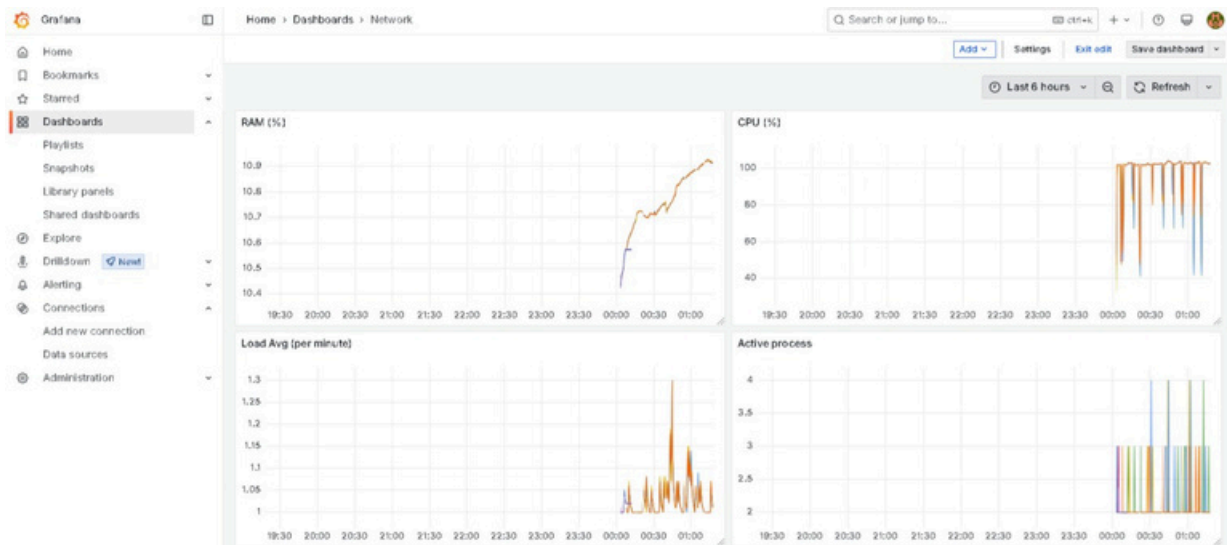


Figure 12. Grafana dashboard

To add a new dashboard in Grafana, a user creates a panel, selects Prometheus as the data source, and writes a query using PromQL. The interface is simple and flexible, allowing changes to time range, chart type, colors, and labels. Figure 13 shows the configuration area for setting up the panel.

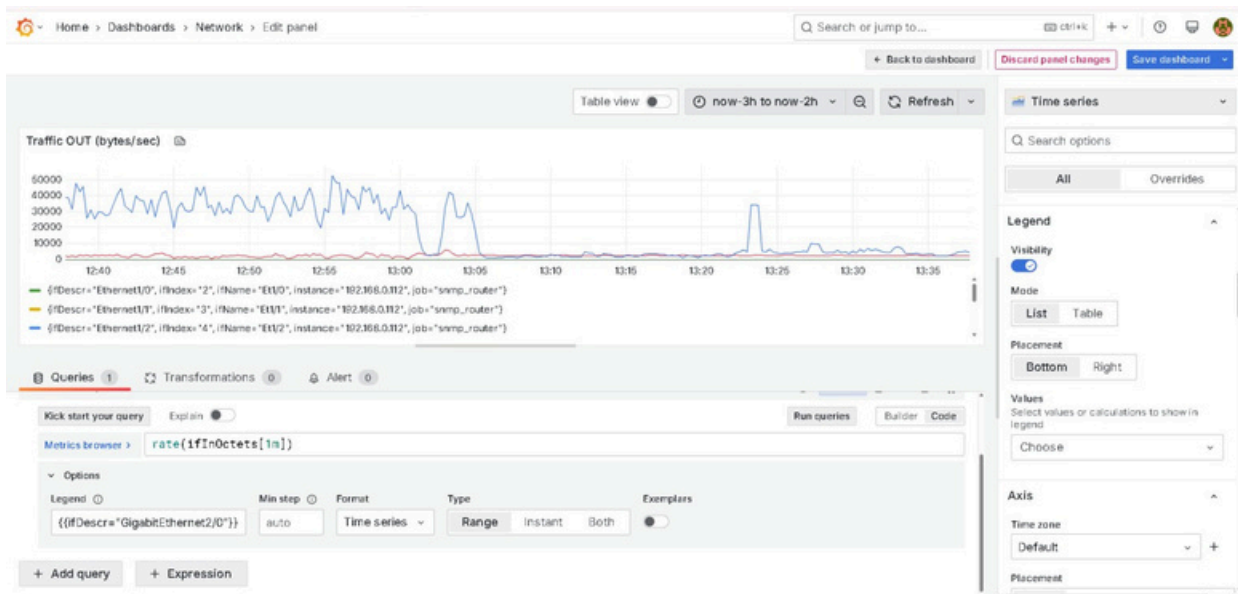


Figure 13. Grafana new dashboard

Prometheus is the most important tool in this setup, as it is responsible for collecting all the metrics from the network devices and passing them to Grafana. These devices include the router, which provides SNMP data, and the containers, which expose metrics using Node Exporter. After Prometheus collects the data, it makes it available for Grafana to be visualized using dashboards and graphs.

4.4 Integration of AI Models for Network Monitoring

To integrate AI scripts into the monitoring system, a batch file named `run.bat` is used. This file runs the Python scripts that train and use AI models to predict future values and detect anomalies. After processing the data, the results are sent to Pushgateway. From there, Prometheus collects the metrics, and they are later visualized in Grafana dashboards [Pro25], [Gra25].

Here is a key part of the script that runs the training and sends data to Pushgateway:

```
:loop
echo [INFO] Running anomaly_detector.py...
python anomaly_detector.py

echo [INFO] Waiting 60 seconds...
ping -n 61 127.0.0.1 > nul

echo [INFO] Running traffic_predictor.py...
python traffic_predictor.py

echo [INFO] Waiting 60 seconds...
ping -n 61 127.0.0.1 > nul

goto loop
```

This part of the script starts an infinite loop that runs two Python files continuously. First, it shows a message in the terminal (`echo [INFO] Running anomaly_detector.py...`) and then runs the

script, `anomaly_detector.py`, that detects unusual patterns in the data using Isolation Forest - a widely recognized method for anomaly detection in network traffic [Xia18]. After that, the script waits for 60 seconds using a fake ping command to the local address 127.0.0.1, which acts as a delay. This address is used because it always responds instantly, ensuring a reliable pause without depending on an external network connection. Once the waiting time is over, the script shows another message and runs the second Python file, `traffic_predictor.py`, which predicts future network traffic using an LSTM model [Zha22], [Gre17], [Cal22]. It waits another 60 seconds before going back to the beginning of the loop and repeating the same steps again.

The AI part of the system uses several important metrics collected through SNMP and provided by Prometheus. These metrics include:

- `ifHCInOctets` - incoming traffic in bytes. Number of received bytes.
- `ifHCOctets` - outgoing traffic in bytes. Number of sent bytes.
- `ifInErrors` - incoming packet errors. Counts the number of errors when receiving packets caused by damaged data or connection issues.
- `ifOutErrors` - outgoing packet errors. Number of errors when sending packets. High values may indicate problems with the network interface or the connection.
- `ifInDiscards` - incoming packets discarded. Measures of how many incoming packets were dropped even without errors, due to full buffers or system limits.
- `ifOutDiscards` - outgoing packets discarded. Shows how many outgoing packets were discarded before being sent, usually because of overload or restrictions.

These metrics are extracted by Prometheus from the router and are then processed by the AI scripts and results generated are sent back to Prometheus using Pushgateway service and can be visualized in Grafana, like any other metric.

The input data for the AI models is fetched using a common Python function

(`fetch_multiple_metrics`) that is shared by both AI scripts (`anomaly_detector.py` and

`traffic_predictor.py`). This function retrieves multiple metrics from Prometheus over a specified time range, then models use them for training.

The function used in both scripts is:

```
def fetch_multiple_metrics(metrics, instance, start, end, step='60s'):
    base_url = 'http://localhost:9090/api/v1/query_range' #prometheus api
    dfs = [] #list to hold dataframes for each metric

    for metric in metrics:
        #build promql query to get rate of metric over 1 minute
        query = f'rate({metric}{{instance="{instance}"}}[1m])'
        params = {'query': query, 'start': start, 'end': end, 'step': step}
        resp = requests.get(base_url, params=params)#request data from
        prometheus
        result = resp.json()

        if 'data' not in result or not result['data']['result']:
            print(f"[ERROR] no data for: {metric}")
            df=pd.DataFrame(columns=["timestamp", metric]) #empty if no data
        else:
            values = result['data']['result'][0]['values']
            df=pd.DataFrame(values,columns=["timestamp", metric])#create df
            df["timestamp"] = pd.to_datetime(df["timestamp"], unit="s")
```

```

df[metric] = df[metric].astype(float) #convert values to float

dfs.append(df) #add df to list

#merge all dataframes on timestamp, keeping all timestamps
df_final = dfs[0]
for df in dfs[1:]:
    df_final = df_final.merge(df, on="timestamp", how="outer")

#sort and drop missing data
df_final = df_final.sort_values("timestamp").dropna()
return df_final

```

This function, `fetch_multiple_metrics`, takes several inputs: a list of metric names (`metrics`), the target instance (`instance`), a time range defined by start (`start`) and end (`end`) timestamps, and an optional step interval (`step`), which defaults to 60 seconds. The `step` parameter controls the time gap between each data point fetched from Prometheus.

For each metric in the list (`metrics`), the function builds a Prometheus query that calculates the rate of change of that metric's value over one minute period. This means instead of fetching the total value of the metric, the function gets how quickly the metric is increasing or decreasing at each moment, measured over one minute window. It then sends an HTTP GET request to Prometheus's `query_range` API endpoint (`base_url`), requesting data within the specified time range (start to end) at intervals defined by `step` [Pro25].

The JSON response from Prometheus is parsed, and the function checks if data is available for the metric. If no data is found, an empty pandas DataFrame is created for that metric with timestamp and metric columns. Otherwise, the time series data (values) is extracted and converted into a pandas DataFrame (`df`), with timestamps converted from Unix time to readable datetime format, and metric values cast to floats so they can be used in mathematical operations.

Each metric's DataFrame is appended to a list (`dfs`). After processing all the metrics, the function combines all the individual DataFrames into one final DataFrame (`df_final`). The merger is done using the timestamp column, so that all metric values are aligned by time. It keeps only the rows where all metrics have valid data, removing any rows with missing values. The final DataFrame is sorted by timestamp to ensure chronological order.

Once the AI models generate predictions or anomaly scores, these results need to be sent back to Prometheus for monitoring and visualization in Grafana. This is done using a function, called `push_metrics`, that pushes the results to the Pushgateway. Prometheus then scrapes the data from Pushgateway, allowing the metrics to be visualized in Grafana.

The function used to send metrics is:

```

def push_metrics(metrics: dict):

    url = "http://localhost:9091/metrics/job/ai_model" #pushgateway endpoint
    body = ""
    for name, value in metrics.items():
        body += f"{name} {value}\n" #format each metric line

    try:
        response = requests.post(url, data=body) #send HTTP POST request
        response.raise_for_status() #raise error for bad response

```

```

#success log
print(f"[PUSH] Sent metrics: {list(metrics.keys())}")
except requests.RequestException as e:
    print(f"[ERROR] Failed to push metrics: {e}") #error log

```

This function takes a dictionary (`metrics`), where each key represents the name of a metric (for example, "anomaly_score"), and each corresponding value is the numerical result produced by the AI model for that metric (such as 0.87). All these metrics are sent to the Pushgateway in a single HTTP POST request.

The function starts by constructing the request body (`body`) as a plain text string. For each metric in the dictionary, it creates a line in the format `metric_name:value`, as required by Pushgateway. All these lines are joined into one string. For example, if the dictionary contains one metric like `{"cpu_usage": 75.3}`, the message sent will be `"cpu_usage 75.3"`.

The target URL for the request is `"http://localhost:9091/metrics/job/ai_model"`, where "ai_model" acts as a custom job label in Prometheus. The function then uses the requests Python library to send the POST request containing all metric values.

If the request is successful, a message is printed listing the names of the metrics that were sent. If any error occurs during the request (e.g., network issue, wrong format), the exception is caught, and an error message is displayed.

These two functions are used in both AI scripts - LSTM and Isolation Forest - to handle data flow. The fetch function brings in the metrics from Prometheus as input for the models, while the push function sends the AI results back to the Pushgateway. Next, we look at the scripts that train the models and send their results to be shown in Grafana.

The traffic prediction script (`traffic_predictor.py`) uses network data collected from the Cisco router running inside the GNS3 virtual topology [Fon18; GNS25] to predict the next value of incoming network traffic. The data is obtained with the function previously explained (`fetch_multiple_metrics`), which fetches several SNMP metrics as input features for the prediction model: incoming octets (`ifHCInOctets`), outgoing octets (`ifHCOutOctets`), input errors (`ifInErrors`), output errors (`ifOutErrors`), input discards (`ifInDiscards`), and output discards (`ifOutDiscards`) [OID25].

Next, the script checks if there are at least 11 data points available, because the prediction model needs enough past data to learn patterns. If there isn't enough data, the script stops and shows an error. After confirming enough data, the script uses a MinMax scaler to normalize all the input metrics, which means it scales their values to be between 0 and 1. This scaling is important because it helps the machine learning model train more effectively and avoids being biased by differences in the scale of the metrics [Lyp24; Sal24]. This process is shown in the next block of code:

```

if len(df) < 11:
    print("[ERROR] Not enough clean data.")
    exit(1)

#scaling
scaler = MinMaxScaler()
scaled_values = scaler.fit_transform(df[metrics])
values = scaled_values

```

After scaling, the script transforms the continuous time series data into smaller sequences of 10 consecutive time steps, because the LSTM model works better with sequential input. For each sequence of 10 data points, the script predicts the next value of the incoming octets metric, which represents the amount of incoming traffic, so the model can learn how past traffic affects future traffic. These sequences are returned as two arrays: X containing the input sequences, and y containing the expected next values to predict. The function for this is shown in the script:

```
#creating
    sequences
def create_sequences_multivariate(data, window=10):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i:i+window])
        y.append(data[i+window][0])
    return np.array(X), np.array(y)

X, y = create_sequences_multivariate(values)
```

The idea of sliding windows of sequential data to feed an LSTM model is common practice in time series forecasting and has been demonstrated to improve the ability of LSTM networks to capture temporal dependencies in traffic data [Zha22], [Gre17], [Cal22].

After the input data has been prepared and turned into sequences, it needs to be reshaped into a format that the LSTM model can understand. Specifically, the LSTM expects the data to have three dimensions: (number of samples, time steps, number of features). Reshaping step is necessary for Keras LSTM layers [Cal22]. This helps the model learn how traffic values change over time, by looking at patterns in the previous steps:

```
X = X.reshape((X.shape[0], X.shape[1], X.shape[2]))
```

Next, the script defines and trains a simple LSTM model. The model has one LSTM layer with 32 units, which learns patterns in the time series data, followed by a dense layer with one output, which predicts the next traffic value. The model is compiled using the Adam optimizer and Mean Squared Error (MSE) as the loss function and then, trained for 5 epochs. Using Adam optimizer with MSE loss is a common and effective approach in training regression models on time series data [Gre17; Cal22]:

```
model = tf.keras.models.Sequential([

    tf.keras.layers.LSTM(32,activation='relu',input_shape=(X.shape[1],
    X.shape[2])),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse')
start_time = time.time()
history = model.fit(X, y, epochs=5, verbose=0)
runtime = time.time() - start_time
```

Mean Squared Error is used to measure how well the LSTM model performs. It shows the average differences between the predicted values and the true values. A smaller MSE means the prediction is more accurate. In the script, the MSE is automatically calculated by TensorFlow library during training, using the loss function. This is done in the following line: `model.compile(optimizer='adam', loss='mse')`. After training, the last recorded loss value (which is the MSE) is extracted from the training history: `mse = history.history['loss'][-1]`.

Once training is complete, the model makes a prediction for the last point in the dataset. Both the predicted and the true values are still scaled between 0 and 1, so they need to be converted back to their original scale using the same MinMax scaler. Only the value of ifHCInOctets (incoming traffic) is predicted. The inverse transformation restores the values to their real scale for easy comparison with real network traffic volumes [Lyp24], [Sal24].

```
y_pred_scaled = model.predict(X[-1].reshape(1, X.shape[1], X.shape[2]))
true_value_scaled = y[-1]
dummy = np.zeros((1, len(metrics)))
dummy[0][0] = y_pred_scaled[0][0]
predicted_value = scaler.inverse_transform(dummy)[0][0]
dummy[0][0] = true_value_scaled
true_value = scaler.inverse_transform(dummy)[0][0]
prediction_timestamp = df["timestamp"].iloc[-1]
```

After making the prediction, the script calculates the MSE metric with method explained before. Then, it sends important values to Prometheus using the `push_metrics` function: the predicted traffic, the true traffic, the MSE, and the time the model took to run. These are also saved locally in a CSV file.

So, this script predicts the next value of incoming traffic using data from the Cisco Router inside GNS3. It prepares the data, trains the model, and sends the prediction along with performance metrics to Prometheus and saves them locally.

Continuing, the script for anomaly detection (`anomaly_detector.py`) analyzes network traffic data collected from the Cisco router inside the GNS3 topology. It uses the same SNMP metrics as the traffic prediction script collected using the same data fetching function

```
(fetch_multiple_metrics).
```

The script uses the `StandardScaler` (from `sklearn.preprocessing` library) to normalize the metrics data (`df[metrics]`). Normalization improves the model's detection ability, which helps ML algorithms perform better [Liu23; Xia18].

Next, the script creates an Isolation Forest model (`IsolationForest` from `sklearn.ensemble` library), with a contamination level set to 0.1 (`contamination=0.1`), meaning it assumes about 10% of data could be anomalies [Liu23].

The model trains on the scaled data (`X`) and predicts which points are normal or anomalous. These predictions are stored in the dataframe column "anomaly" (`df["anomaly"]`). The model also assigns an anomaly score to each data point (`df["anomaly_score"]`), indicating how unusual each point is.

The script then extracts useful performance information: the last anomaly prediction is converted into a flag (`anomaly_flag`), where 1 means an anomaly was detected and 0 means normal. It also calculates the overall anomaly rate (`anomaly_rate`), and the average and maximum anomaly scores (`anomaly_score_mean` and `anomaly_score_max`). All these metrics are sent to Prometheus for monitoring and saved locally in a CSV file, similarly to the traffic prediction script [Xia18], [Liu23].

```
scaler = StandardScaler() #scale to have mean=0 and std=1
X = scaler.fit_transform(df[metrics])
model = IsolationForest(contamination=0.1) #10% expected anomalies
df["anomaly"] = model.fit_predict(X)#label pts as normal (1) or anomaly (-1)
df["anomaly_score"] = model.decision_function(X)

last_anomaly = df["anomaly"].iloc[-1] #get anomaly label of last point
anomaly_flag = 1 if last_anomaly == -1 else 0 #flag 1 if last point is anomaly
anomaly_rate = (df["anomaly"] == -1).mean() #overall fraction of anomalies
anomaly_score_mean = df["anomaly_score"].mean() #average anomaly score
anomaly_score_max = df["anomaly_score"].max() #maximum anomaly score
```


5. Experimental Results

This chapter is focused on practical results obtained by testing three different scenarios. Each scenario is built on the previous one with increasing complexity and functionality. The first scenario presents a basic GNS3 topology without being monitored or AI capabilities. The second scenario adds monitoring system, and the last scenario integrates AI features for anomaly detection and traffic prediction.

The purpose of these experiments is to observe how the addition of monitoring tools and intelligent models can improve network visibility and control. Each scenario is analyzed using different types of metrics collected from the network, such as traffic volume, interface activity, system usage, and communication patterns. The results are presented using real examples and visualizations that show the motivation of moving from a simpler to a smarter system.

5.1 Scenario 1: GNS3 Topology without Monitoring and AI Facilities

Without being monitored, this topology (explained in Chapter 4.1 GNS3 Installation with Docker Containers) represents a basic network that can only send or receive traffic from the Internet with a general overview, without knowing what happens behind.

The method to generate traffic (detailed in Chapter 4.1) uses simple Linux-based commands (like ping) to simulate real network activity. An example of such a command (executed from a container) and its output is shown below:

```
/ # ping -c 100 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=117 time=13.0 ms
64 bytes from 8.8.8.8: seq=1 ttl=117 time=12.8 ms
64 bytes from 8.8.8.8: seq=2 ttl=117 time=12.6 ms
64 bytes from 8.8.8.8: seq=3 ttl=117 time=12.7 ms
```

These results show that the container is successfully sending packets to the Internet address 8.8.8.8 (Google's DNS server) and receiving replies. Each line in the output represents a reply for one packet sent, showing details like the size of the reply (64 bytes), the sequence number (seq), the time-to-live (ttl), and the time it took for the packet to travel (time). This basic ping test confirms that the network is working and able to communicate with the outside world.

Another method used to generate traffic involves a special container called webterm, which simulates real user behavior by accessing websites. This container runs a lightweight GUI and uses TightVNC (a remote desktop server) to allow interaction with a browser. TightVNC allows to open and view websites from the container like a normal browser.

This activity creates a more complex and realistic traffic pattern, like how users normally interact with the web. A screenshot showing a page opened in the webterm container is illustrated in Figure 14.

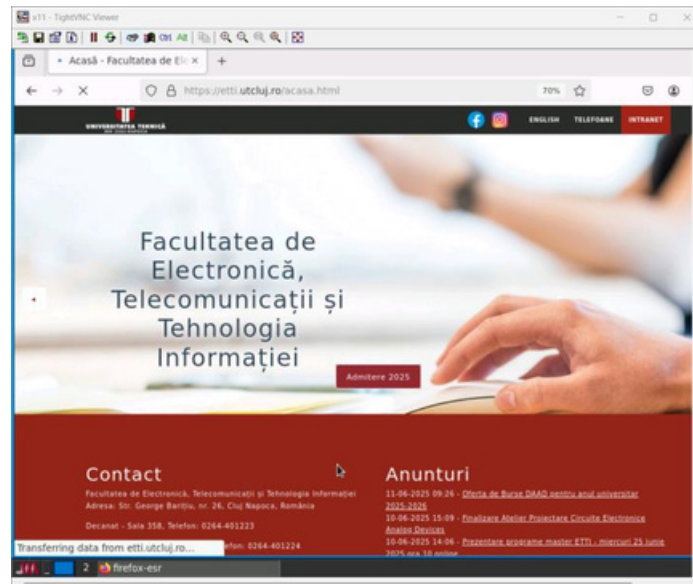


Figure 14. GUI provided by Webterm container

Even though traffic is successfully generated in this topology and responses are received, there is still no visibility into what happens inside the network. For example, it is not possible to see how data flows between the router and the containers, how much traffic each link carries, or which devices are active.

Another important limitation is the lack of history. This setup does not store or display past network activity. For instance, after a ping command, the results are lost and cannot be reviewed later. This makes it difficult to analyze the network's performance over time or to identify any issues.

There are also no alerts or automatic ways to detect problems. If a device becomes disconnected, a link becomes down, or an error occurs, there is no notification or log to show what happened. All checks must be done manually, using basic commands. Furthermore, no traffic patterns can be observed, so there is no way to tell if network usage increases or if unusual behavior appears. Issues like congestion or security risks cannot be detected in this form.

These limitations show that the basic setup is only useful for simple connectivity tests and educational purposes. It confirms that the devices are correctly connected and that the Internet works, but it does not provide any detailed insight into network performance or behavior. For that, monitoring tools are needed, and they are introduced in the next scenario.

5.2 Scenario 2: GNS3 Topology with Monitoring Capabilities

In this scenario, the same basic topology is extended with monitoring tools like Prometheus, Grafana, SNMP and Node Exporter, allowing us to collect and visualize detailed network metrics. The installation and configuration of these tools were described in Chapter 4. This section presents the results they provide, including various metrics collected from the router and containers inside GNS3.

Prometheus collects many types of data from the containers using Node Exporter. One of the most useful categories includes network traffic metrics which show how much data is received or sent

by each container over time. Below is an example of raw metric values collected from a container using Node Exporter, but many other metrics are provided by this service:

```
# TYPE node_network_receive_bytes_total counter
node_network_receive_bytes_total{device="eth0"} 25528
node_network_receive_bytes_total{device="lo"} 0
# TYPE node_network_transmit_bytes_total counter
node_network_transmit_bytes_total{device="eth0"} 193653
node_network_transmit_bytes_total{device="lo"} 0
```

These results show the total number of bytes received and sent by each network interface on the container. In this case, the container has two interfaces: eth0, which is the main interface used to communicate with the router and other devices, and lo, which is the local loopback interface used internally by the container.

- `node_network_receive_bytes_total{device="eth0"} 25528` means the container received 25,528 bytes through eth0.
- `node_network_transmit_bytes_total{device="eth0"} 193653` shows it sent 193,653 bytes through the same interface.
- The lo interface shows 0 bytes, because it is not used for external communication in this setup.

These metrics are taken from a single container, but each container in the topology exposes these metrics using its own Node Exporter service. Prometheus then collects this data and Grafana displays it.

One example of such a result is shown in Figure 15, where a Prometheus query (`rate(ifHcOutOctets[5m])`) displays the outgoing traffic on all interfaces of the router. These metrics are collected using SNMP (running on router). Only two interfaces have visible traffic, while the others show a value of 0. This happens because FastEthernet0/0 is connected to the containers, where most of the local communication happens, and GigabitEthernet2/0 is the interface connected to the Cloud node, which provides Internet access, so, all the traffic going outside the GNS3 network passes through this interface.

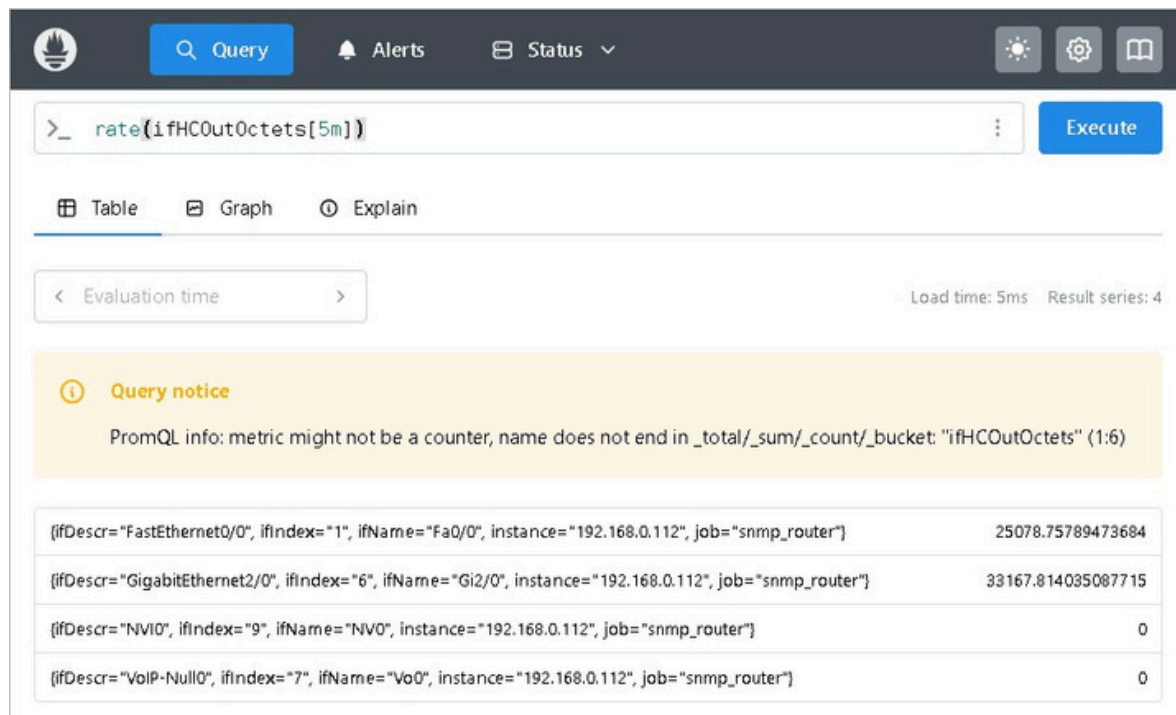


Figure 15. Prometheus query showing traffic per router interface

To visualize the metrics provided by Prometheus, Grafana is used. It connects to Prometheus and makes it easy to create custom dashboards that include all the important graphs. These dashboards allow users to follow the network's behavior over time, identify trends, and quickly notice if something unusual is happening.

Figure 16 provides an example of how Grafana can be used to show different metrics collected from both the router and the containers. The image contains two dashboards, each built using a different query.

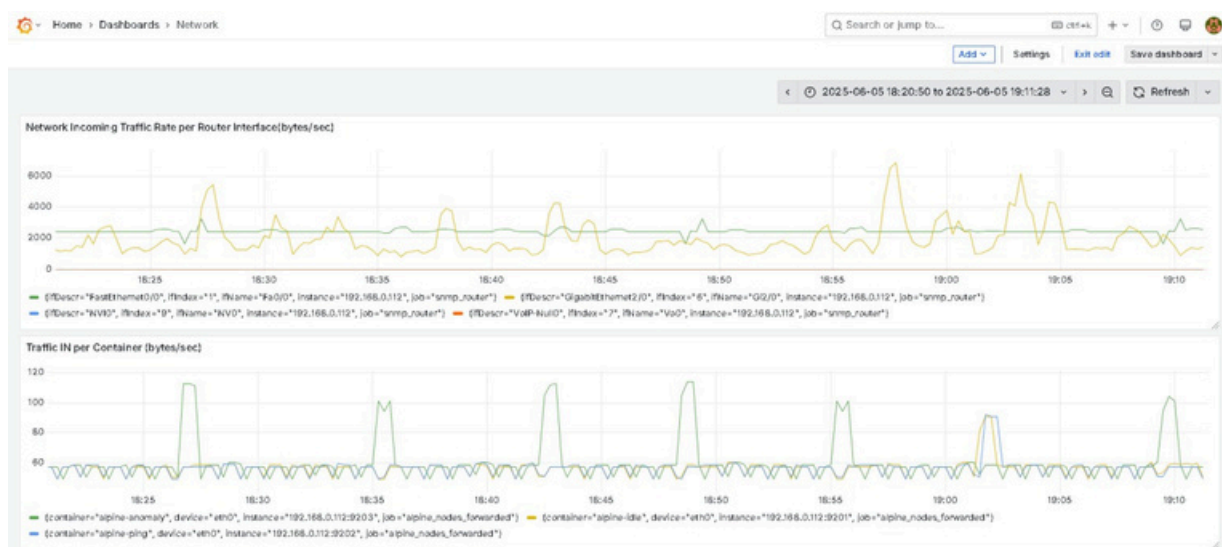


Figure 16. Visualization of network metrics in Grafana

The capture shows two Grafana dashboards displaying network traffic metrics collected with different queries. The first dashboard uses the query `rate(ifHCInOctets[1m])` which displays

the inbound traffic on the router's interfaces collected via SNMP. Only two interfaces (FastEthernet0/0 between containers and router and GigabitEthernet2/0 that connects the topology to Internet) show active traffic, while the other interfaces have zero traffic because they are not currently in use. The second dashboard is based on the `rate(node_network_receive_bytes_total{device!="lo"}[1m])`, which provides a view of the network traffic received by the container's network interfaces, measured in bytes per second. Here are just a few example graphs Grafana can show. It can display any data that Prometheus collects. Both dashboards show how powerful Grafana is when used with Prometheus. They provide useful insights about the current state of the network and help understand how traffic flows through the topology. These two examples are just a small part of what Grafana can display, but any metric that Prometheus collects can be shown.

In addition, Prometheus stores all metrics as time-series data, which means that every value is saved along with a timestamp. During testing, it was observed that Prometheus keeps historical data for at least 30 days, which is useful for long-term monitoring. For example, Figure 15 shows a graph based on past data stored in Prometheus. This ability to view data from previous days or weeks helps identify patterns, compare behavior across time, and detect problems that may develop slowly.

5.3 Scenario 3: Network Topology with Monitoring and AI Integration

The last scenario is a combination of the previous two with AI capabilities added. Pushgateway, an extra tool, is needed to send results from AI scripts to Prometheus and then to Grafana for visualizations.

The goal is to not only collect and visualize network traffic using Grafana and Prometheus, but also to analyze the traffic data using machine learning models. These models are used to predict traffic values and detect anomalies.

For prediction feature, the script collects previous past values so the model can predict future network traffic. A Grafana graph displayed in Figure 17 is used to compare the real traffic values with the predicted ones. The predicted traffic is displayed on the same timeline as the actual traffic, which helps to clearly see how well the model performs.



Figure 17. Traffic prediction visualization in Grafana

The prediction script runs every 60 seconds and sends results to Pushgateway, which makes them available in Prometheus. In the Grafana graph, the predicted traffic line does not match the real traffic line exactly at the beginning. This is because the LSTM model is trained from scratch each time the script runs, using the most recent 10 minutes of network data. This small amount of data

is not enough for the model to fully learn the traffic patterns, and, as a result, its predictions are less accurate in the first few minutes. However, after around 10 minutes of running, the model begins to recognize the pattern better, and the predicted values start to follow the real traffic. This can be observed in the graph in Figure 17, where after the initial learning period from the beginning, the two lines (real and predicted traffic) almost overlap. So, the delay and differences at the beginning are normal due to limited data and the model improves quickly after more data becomes available.

To better understand the model's performance over time, Figure 18 shows a graph of the MSE values recorded at each prediction result. This graph is synchronized in time with the one displaying predicted vs. real traffic values in Figure 17, and it shows how prediction accuracy changes.

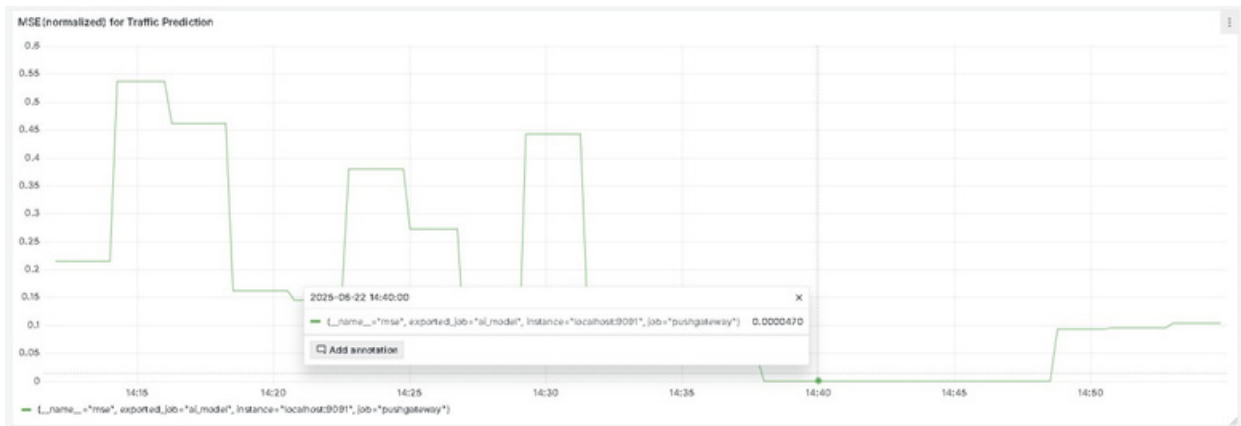


Figure 18. MSE for Traffic prediction in Grafana

In the first few minutes after starting the system, the MSE is higher because the LSTM model has access to only a small amount of data (less than 10 minutes) and has not yet learned the full traffic pattern. However, as more data becomes available and the model retrains at every 60 seconds, it begins to make better predictions. After around 10 minutes of running, the MSE drops significantly, reaching very small values such as 0.00005, and this happens at the same time the prediction and real traffic lines begin to overlap (Figure 17). The graph confirms that once the model has enough historical data, it becomes more accurate and stable in predicting traffic. Table 2 shows how important prediction data is saved locally in a CSV file after each script run for historical analysis, debugging, and future improvement. It also used to compare results, generate reports, or retrain with saveddata.

Timestamp	Predicted value	True value	MSE	Runtime [s]
2025-06-22 14:11:46.995304	719,57	2878,76	0,22	1,74
2025-06-22 14:13:56.998888	458,46	3137,93	0,54	1,74
2025-06-22 14:16:06.683941	699,88	3964,93	0,46	1,74
2025-06-22 14:18:16.327202	808,73	2863,56	0,16	1,48
2025-06-22 14:20:25.938472	1613,27	3956,89	0,15	1,51
2025-06-22 14:22:35.213808	128,01	3967,53	0,38	1,52
2025-06-22 14:24:43.838752	890,59	3362,22	0,27	1,35
2025-06-22 14:26:53.157579	2791,97	2984,71	0,06	1,51
2025-06-22 14:29:02.865396	3223,51	3970,98	0,44	1,50
2025-06-22 14:31:12.162633	2866,52	2856,56	0,14	1,49
2025-06-22 14:33:21.825702	2885,70	2855,16	0,13	1,42
2025-06-22 14:35:31.384436	3980,65	3965,40	0,04	1,47
2025-06-22 14:37:41.129955	2876,77	2854,67	0,00047	1,44

2025-06-22 14:39:50.949118	2859,94	2855,24	0,00005	1,50
2025-06-22 14:42:00.480202	3966,30	3962,71	0,00003	1,48
2025-06-22 14:44:10.105495	2856,07	2857,00	0,00003	1,50
2025-06-22 14:46:19.117623	2860,08	2858,04	0,00007	1,41
2025-06-22 14:48:27.824072	4298,59	5052,27	0,09339	1,37

Table 2. Recorded prediction results, realtraffic, MSE, timestamps and runtime for LSTM

Each row represents the output of one prediction cycle – collect and preprocess data, train the model, send results. The Timestamp column indicates the moment when the results were saved, using the local time of the machine. The Predicted value and True value columns represent the traffic in bytes, one predicted by the LSTM model, the other collected directly from Prometheus. The MSE show the accuracy of prediction – a lower value means better performance. The Runtime [s] indicates how much time it took for the model to train and generate a prediction.

This data, saved in a CSV file, are also reflected in Figures 17 and 18 where the same data is displayed through Grafana graphs to make them easier to understand and visualize.

To detect anomalous traffic, the script processes the latest 10 minutes of network data (using metrics presented above) and it sends performance metrics (anomaly_score_mean, and anomaly_score_max) to Prometheus and then to Grafana for visualization. These values are also stored in a local CSV file for future analysis.

Figure 19 shows how anomalies were visualized in Grafana dashboards using a graph with a True/False scale, 'True' indicating the anomaly occurred and 'False' representing the normal traffic.

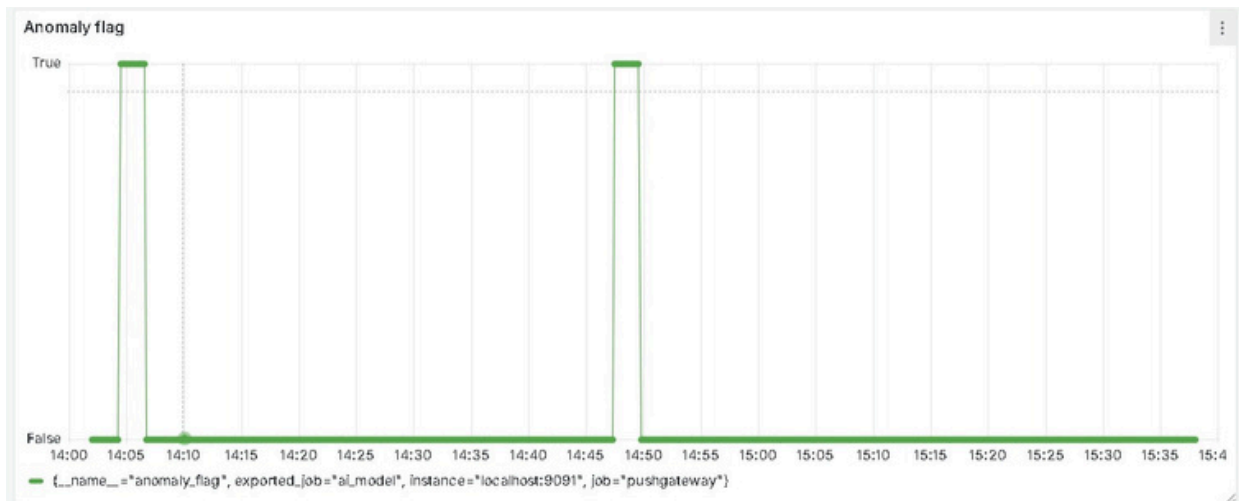


Figure 19. Anomaly flags visualized in Grafana dashboard

These flags were generated by the anomaly detection script and pushed to Prometheus every 60 seconds using Pushgateway. Because Prometheus collected data at regular intervals (every 15 seconds), the same anomaly flag was recorded multiple times between script executions. For example, if the script detected an anomaly, that 'True' value might be scraped and stored four times before the next update. This resulted in a short series of identical values in the data, even though the script only produced one result.

Figure 20 shows three performance metrics used to evaluate the anomaly detection system: `anomaly_rate`, `anomaly_score_mean`, and `anomaly_score_max`.

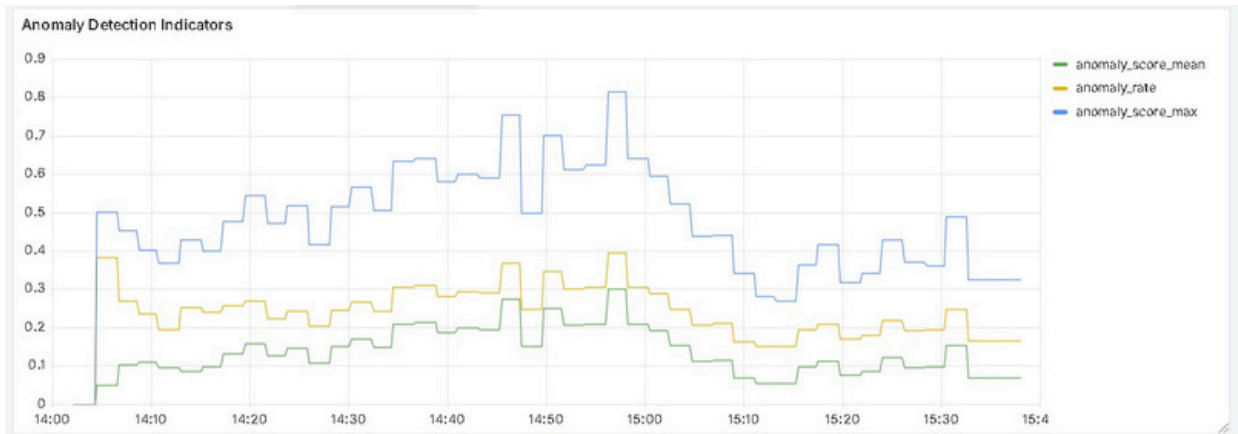


Figure 20. Performance metrics for anomaly detection system displayed in Grafana

These values are used to understand how often the system detects unusual activity and how strong those anomalies are:

- `anomaly_rate` shows how many data points (out of all the collected ones) were identified as anomalies. For example, an anomaly rate of 0.2 means that 20% of the recent data points were marked as unusual or different from normal behavior.
- `anomaly_score_mean` is the average score given to all data points by the Isolation Forest model. Each point gets a score showing how unusual it is. A lower score means it is more likely to be an anomaly. By taking the average of all these scores, we can see how "normal" or "abnormal" the overall system behavior was during the last check.
- `anomaly_score_max` is the highest score among all the points. A high score means the points are normal. If this value is close to 0.5 or higher, it means the system has at least one normal point, even if other points may be less normal.

The graphs appeared in steps because the values were updated every minute, while Prometheus collected data every 15 seconds. This caused the same value to be shown multiple times on the graph, creating a staircase shape. Each metric line followed a similar direction and shape, which shows that the system is consistent in detecting anomalies. The average anomaly score was lower (under 0.3) because the network traffic was varied when generating, especially from the 'Anomaly container'. Also, the Isolation Forest model was configured with a contamination rate of 0.1, which meant that 10% of the data points were considered anomalous. This made the model more sensitive to changes.

This visualization of the anomaly detection system's results in Grafana also made it easier to follow and to understand how the system reacted to unusual traffic.

6. Conclusions

The main goal of this thesis was to build a complete system that can monitor and analyze traffic in any network that uses a router, working even without an Internet connection, but this includes some limitations. The system was designed to collect data in real time and visualize them to understand the current state, helping to track the traffic, to check the device activity, and to detect issues earlier. By using this system, it became much easier to manage network resources and to detect if some problems occur.

Another important part of this thesis was the integration of AI components to the system so that we do not just observe the network. Artificial Intelligence added two main features: (1) detection of anomalies in the network; and (2) prediction of future traffic. The first helps to find strange or unusual activity caused by errors, misconfigurations, or security threats. The second one helps to anticipate problems before they appear, such as congestion or system overload. These functions give more time to react and to prevent issues that can affect network performance.

This monitoring system was built using only open-source tools because they were free and easy to integrate. Another important reason is the large community support for these tools, which contributes to solving the problems quickly and improving the system over time. Also, they are flexible and customizable, allowing them to be adapted for different network purposes.

Currently, there are some commercial solutions on the market, such as SolarWinds, PRTG, or Datadog, similar to ours. However, they cost money, they have usage limits, or they are not fully customizable. Unlike those, the thesis proposed a solution fully transparent, easy to modify, and does not rely on cloud or paid services. It also includes AI-based processing, which is usually not available in the standard monitoring tools without additional costs.

Future improvements of this thesis can focus on both the AI part and the network configuration. The current LSTM model is trained from scratch every time using only 10 minutes of data, limiting the accuracy of traffic prediction, so, a better approach would be to train the model on more data, save it, and update it over time. Also, more advanced models or longer training sequences could improve results. For anomaly detection, adjusting the contamination level or combining different models might reduce false alerts. On the network side, the topology can also be extended by adding new devices that generate more and different types of traffic, such as DNS, file transfers, or video streams, testing the system under more realistic conditions. Another option is to test other AI models (like GRU, CNN, or Random Forest) and compare their performance with the current used models, to understand which is more suitable for real-time traffic prediction or anomaly detection in this type of network. On the network side, the GNS3 topology can be expanded by adding new types of devices, such as DNS servers, file transfer nodes, or media streaming containers, to simulate a wider range of real-world traffic.

Generative AI tools were used to improve the clarity and readability of the text. Specifically, it was applied in sections such as Theoretical Fundamentals and Implementation for rephrasing, polishing, and enhancing expression. All content was carefully reviewed to ensure accuracy and originality.

7. References

- [Alb22] L. Alberro, A. Castro, E. Grampin, "Experimentation Environments for Data Center Routing Protocols: A Comprehensive Review", *Future Internet*. 2022; 14(1):29. <https://doi.org/10.3390/fi14010029>
- [Alt24] E. Altulaihan, M. A. Almaiah și A. Aljughaiman, "Anomaly Detection IDS for Detecting DoS Attacks in IoT Networks Based on Machine Learning Algorithms," *Sensors*, vol. 24, art. nr. 713, 2024, doi: 10.3390/s24020713
- [Cal22] O. Calzone, "An Intuitive Explanation of LSTM," *Medium*, Feb. 21, 2022. [Online]. Available: <https://medium.com/@ottavio.calzone/an-intuitive-explanation-of-lstm-a035eb6ab42c>
- [Cir25] S.D. Cîrnat, V. Dobrota, "Isolation Forest Anomaly Detection and Long Short-Term Memory Traffic Prediction", 20th Student Symposium on Electronics and Telecommunications SSET 2025, Technical University of Cluj-Napoca, 2025.
- [Emm23] I.D. Emmanuel, N. Linge, and S. Hill, "Analysis of SD-WAN Packets using Machine Learning Algorithm," 2023 Conference on Communications Technology and Society (ICTAS), Durban, South Africa, 2023, pp. 1-6, doi: 10.1109/ICTAS56421.2023.10082743.
- [Fon18] M. Fomin, "GNS3 for Network Emulation", *International Journal of Engineering and Technology*, vol. 7, no. 4, pp. 317–320, 2018.
- [GNS25] "GNS3 — Graphical Network Simulator," GNS3.com, 2025, [Online]. Available: <https://www.gns3.com>
- [Gra25] "Introduction to Grafana," Grafana Documentation, 2025, [Online], Available: <https://grafana.com/docs/grafana/latest/>.
- [Gre17] D. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A Search Space Odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017, doi: 10.1109/TNNLS.2016.2582924.
- [Jai24] M. Jain și A. Shah, "Anomaly Detection Using Convolutional Neural Networks," *ESP International Journal of Advancements in Computational Technology*, vol. 2, nr. 3, pp. 12–22, 2024, doi: 10.56472/25838628/IJACT-V2I3P102
- [Lal20] B. Lal, A. Patel, R. Sharma and R. Thakur, "Survey of Anomaly Detection Techniques in Data Mining", *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, vol. 9, no. 1, pp. 3799–3803, Nov. 2019.
- [Liu23] Y. Liu, S. Zhang, and L. Song, "Learning to Improve Isolation Forest," *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)*, held in Macao, S.A.R., 2023, pp. 2383–2390. International Joint Conferences on Artificial Intelligence Organization. [Online]. Available: <https://www.ijcai.org/proceedings/2023/0264.pdf>
- [Lyp24] A. Lypka, D. Malowany, and J. Domanska, "Benchmarking Feature Extraction Tools for Network Traffic Classification," *Scientific Reports*, vol. 14, no. 1, Apr. 2024, Article no. 70983. doi: 10.1038/s41598-024-70983-6.

- [OID25] "Simple Network Management Protocol: OID Overview and MIBs", Global OID Reference Database, 2025, [Online], Available: <https://oidref.com>
- [Pou24a] N. Poulton, "Docker Deep Dive. Zero to Docker in a single book!", Nigel Poulton and Lean Publishing, 2024.
- [Pou24b] N. Poulton, "The Kubernetes Book", Nigel Poulton and Lean Publishing, 2024.
- [Pro25] "Prometheus: Monitoring system and time series database," Prometheus.io, [Online]. Available: <https://prometheus.io/>.
- [Pyn25] "Top 10 Most Popular Network Simulation Tools (2025)," PyNet Labs, 2025, [Online], Available: <https://www.pynetlabs.com/top-most-popular-network-simulation-tools/>.
- [Sal24] A. A. Salahuddin, M. F. Bari, I. A. Qazi, and R. Boutaba, "Machine Learning for Classification of SDN Traffic," arXiv preprint, arXiv:2501.13004, Jan. 2024. [Online]. Available: <https://arxiv.org/abs/2501.13004>
- [Tur19] Y. Turk, E. Zeydan and Z. Bilgin, "A Machine Learning Based Management System for Network Services," 2019 International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), Barcelona, Spain, 2019, pp. 1-8, doi: 10.1109/WiMOB.2019.8923572.
- [Tur24] F. Turkal, "How does Docker ACTUALLY work? The Hard Way: A Comprehensive Technical Deep Diving," Medium, Jan. 4, 2024. [Online]. Available: <https://medium.com/@furkan.turkal/how-does-docker-actually-work-the-hard-way-a-technical-deep-diving-c5b8ea2f0422>
- [Wen21] Y. Wen, Y. Liu, C. Lin and J. Pan, "Point-Denoise: Unsupervised Outlier Detection for 3D Point Clouds Enhancement," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021, pp. 14660–14669, doi: 10.1109/CVPR46437.2021.01444.
- [Wik25a] "Long short-term memory," Wikipedia, The Free Encyclopedia, 2025, [Online]. Available: https://en.wikipedia.org/wiki/Long_short-term_memory
- [Wik25b] "Isolation Forest," Wikipedia, The Free Encyclopedia, 2025, [Online]. Available: https://en.wikipedia.org/wiki/Isolation_forest
- [Xia18] C.-H. Xiao, C. Su, C.-X. Bao and X. Li, "Anomaly Detection in Network Management System Based on Isolation Forest," 2018 4th Annual International Conference on Network and Information Systems for Computers (ICNISC), Webjiajie, China, 2018, pp. 561-60, doi: 10.1109/ICNISC.2018.0002022, doi: 10.1051/mateconf/202235502007.
- [Zha22] J. Zhao, X. He, "NTAM-LSTM Models of Network Traffic Prediction," MATEC A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023, [Online]. Available: <https://d2l.ai>

Isolation Forest Anomaly Detection and Long Short-Term Memory Traffic Prediction

Stefan-Darian Cirnat
Communications Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
Cirnat.Ov.Stefan@student.utcluj.ro

Virgil Dobrota
Communications Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
Virgil.Dobrota@com.utcluj.ro

Abstract— The integration of Artificial Intelligence (AI) with network traffic analysis offers a new approach to monitor and to predict network behavior. By leveraging Grafana, Prometheus and Node Exporter for real-time data collection and visualization, the system enhances traditional monitoring methods. These tools play a crucial role in improving network management: Prometheus collects and stores network data, while Node exporter delivers detailed metrics on endpoint devices and Grafana comes with an advanced visual interface, enabling efficient traffic analysis and monitoring. Machine Learning models were used for anomaly detection and traffic forecasting to identify potential issues and to predict the network conditions. By combining these technologies, the system enabled real-time monitoring, while also supporting anomaly detection and traffic prediction, thereby improving network performance and optimizing resource allocation.

Keywords— artificial intelligence, Grafana, machine learning network traffic analysis, Prometheus.

I. INTRODUCTION

In the context of today's rapidly advancing technologies, the increasing reliance on networked systems has highlighted the need for advanced solutions that offer better monitoring, management, and prediction of network behavior [1], [2]. One such solution comes in the form of network traffic monitoring and analysis systems, which provide real-time insights into network performance [6]. Traditional network monitoring methods, while effective, often fall short in terms of proactive management and predictive capabilities [3]. To address these limitations, this project integrates Grafana, Prometheus, and Node Exporter for real-time data collection and visualization [4], combining them with Artificial Intelligence (AI) for anomaly detection and traffic prediction [2], [5]. By leveraging these technologies, the system enhances traditional monitoring by offering more precise forecasting, improving network performance, and optimizing resource allocation [1], [2]. The use of AI in this context enables the identification of potential issues before they escalate, offering a smarter, more dynamic approach to network management [3], [5]. This approach represents a significant step towards making network traffic analysis more predictive and efficient [2], [6].

This paper is structured as follows: Section II presents the proposed intelligent network monitoring architecture, which integrates Prometheus and Grafana with AI-based modules for traffic prediction and anomaly detection. Section III describes the implementation details and the experimental setup in GNS3 emulator. The paper ends with conclusions and future work.

II. NETWORK MONITORING ARCHITECTURE

The proposed architecture is divided into three sections, as in Fig. 1.

Network topology – represent the monitored topology which includes Docker containers that simulate real endpoints where metrics are transferred to the next section through a router [4].

Data Collection & Visualization is the place where metrics are processed to obtain relevant graphs for monitoring desired network and these metrics are also passed to AI models' section to predict traffic patterns and detect anomalies [6], [7].

Each *Docker container* has a Node Extractor server running on it which provides a comprehensive set of metrics about the status of system resources [6]. These metrics are essential for monitoring performance and detecting problems in network infrastructure [1].

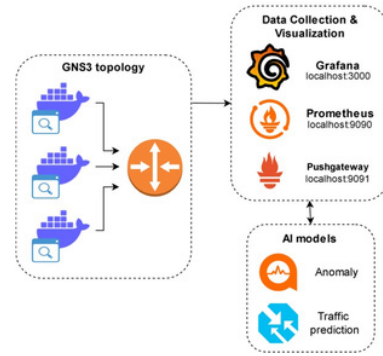


Fig. 1. Intelligent Monitoring System using Prometheus, Grafana, and AI

The Data Collection & Visualization component serves as the central point for aggregating and presenting network metrics. It is composed of three key tools: (a) Prometheus: a powerful time-series database used to collect and store metrics from network devices and systems [6]; (b) Pushgateway: a service that receives custom metrics (such as predictions and anomaly flags) pushed by external AI scripts; and (c) Grafana: an open-source analytics and visualization platform used to create dashboards that display these metrics stored in Prometheus in real time [7]. Prometheus scrapes metrics both from the Simple Network Management Protocol (SNMP) - enabled router and from Node Exporter running on the containers, while also pulling pushed data from Pushgateway [6]. Grafana then provides intuitive visual

insights into system health, network performance, and potential anomalies [7]. The AI models section introduces advanced data-driven components designed to enhance traditional network monitoring. These models process the time-series collected by Prometheus to identify patterns, detect irregularities, and anticipate future behavior [1],[3]. This section includes two core functionalities: *Anomaly Detection*: a model trained on historical data to learn what constitutes normal behavior in the network (e.g. CPU usage, traffic flow, packet drops) [1],[3]; *Traffic Prediction*: a model that forecast future network load based on trends observed in past metrics, using DL models such as Long Short-Term Memory (LSTM) [2],[10].

III. IMPLEMENTATION AND EXPERIMENTAL RESULTS

The test environment uses GNS3 emulator to integrate virtual devices and Docker containers into a controlled, reproducible topology for both standard and AI-driven monitoring simulating real-world conditions [4].

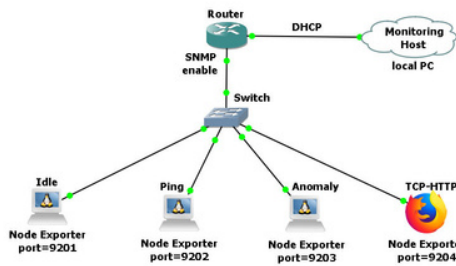


Fig.2. GNS3-based network topology

The GNS3 topology links Docker containers (each running Node Exporter) to a Cisco router with SNMP enabled. It uses Dynamic Host Configuration Protocol (DHCP) to connect to the cloud (local PC) and it performs port forwarding, allowing Prometheus to access container metrics [4],[5]. On the PC, Prometheus, Pushgateway, and Grafana handle real-time metric collection, anomaly detection, and traffic prediction [6],[7]. To better illustrate the AI mechanisms see the following key code snippets: (where -l=anomaly and

Isolation Forest anomaly detection

```
model = IsolationForest(contamination=0.1)
df["anomaly"] = model.fit_predict(X)
```

LSTM traffic prediction:

```
model = tf.keras.models.Sequential([
    tf.keras.layers.LSTM(32, activation='relu', input_shape=(X.shape[1], 1)),
    tf.keras.layers.Dense(1)
])
model.fit(X, y, epochs=5, verbose=0)
y_pred = model.predict(X[-1].reshape(1, X.shape[1], 1))
```

This builds and trains an LSTM neural network to learn traffic patterns and predict the next traffic value based on past data sequences [2], [10]. The traffic prediction graph (see Fig.3) displays the system's forecasted network values over time. Negative values can appear due to model limitations or input noise, highlighting moments where predictions diverge and pointing to potential inaccuracies or unusual patterns [3]. The model runs automatically at set intervals, retraining from scratch each time on the latest data and generating new predictions, ensuring it constantly adapts to recent network behavior [1],[2].

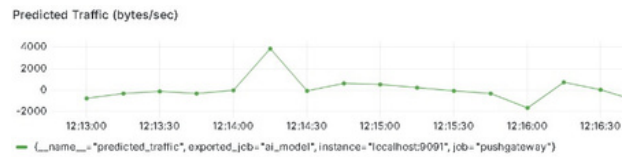


Fig.3. Traffic prediction over time

The automation mechanism built into this project plays a crucial role in ensuring continuous and up-to-date monitoring. Specifically, the system is designed to automatically execute both the anomaly detection and traffic prediction scripts at fixed time intervals (e.g., every 60 seconds) without requiring manual intervention. Each time the scripts run, they collect the latest metric data from Prometheus, retrain their respective models (Isolation Forest for anomalies, LSTM for predictions) on the most recent dataset, and then generate updated outputs. These outputs, anomaly flags and predicted traffic values, are automatically pushed to the Pushgateway, from where Prometheus picks them up for visualization in Grafana.

IV. CONCLUSIONS AND FUTURE WORK

This project successfully demonstrates the integration of advanced AI techniques into a virtualized GNS3 network environment, combining real-time data collection, efficient metric aggregation, and dynamic visualization. By designing an automated system that continuously trains anomaly detection and traffic prediction models, the setup not only replicates realistic network behavior but also delivers actionable insights into system performance and potential issues. The use of Docker containers and port-forwarded SNMP metrics ensures modularity and flexibility, making the solution scalable for future extensions.

REFERENCES

- [1] A.O.Salau and M.M. Beyene, "Software defined networking based network traffic classification using machine learning techniques," Scientific Reports, vol. 14, no. 20060, 2024. doi: 10.1038/s41598-024-70983-6.
- [2] Y. Turk, E. Zeydan, and Z. Bilgin, "A Machine Learning Based Management System for Network Services," in 2019 Int. Conf. on Wireless and Mobile Computing, Networking and Communications (WiMob), Barcelona, Spain, 2019, pp. 1-8. doi: 10.1109/WiMOB.2019.8923572.
- [3] B. Lypa, I. Horyn, N. Zagorodna, D. Tymoshchuk, and T. Lechachenko, "Comparison of Feature Extraction Tools for Network Traffic Data," CEUR Workshop Proceedings, ITTAP 2024, vol. 3373, pp. 486-506.
- [4] D.J. Fonseca and W.Z. Zheng, "Using GNS3 to enhance the practical learning of computer network design and implementation," International Journal of Engineering Education, vol. 34, no. 4, pp. 1332-1342, 2018.
- [5] I.D. Emmanuel, N. Linge, and S. Hill, "Analysis of SD-WAN Packets using Machine Learning Algorithm," in 2023 Conf. on Information Communications Technology and Society (ICTAS), Durban, South Africa, 2023, pp. 1-6. doi: 10.1109/ICTAS56421.2023.10082743.
- [6] "Prometheus - Monitoring system and time series database," Prometheus.io, 2025, [Online]. Available: <https://prometheus.io>.
- [7] "Grafana - The open observability platform," Grafana Labs, 2025, [Online]. Available: <https://grafana.com>.
- [8] "Node Exporter," Prometheus.io, 2025, [Online]. Available: <https://prometheus.io/docs/guides/node-exporter>.
- [9] "Pushgateway," Prometheus.io, 2025 [Online]. Available: <https://prometheus.io/docs/practices/pushing>.
- [10] F. Chollet, "Keras Documentation," Keras.io, 2025 [Online]. Available: <https://keras.io>.

THE 20th STUDENT SYMPOSIUM ON
ELECTRONICS AND TELECOMMUNICATIONS
May 23, 2025

1st Special Award

Ștefan-Darian CÎRNAT

author of the paper

"Isolation Forest Anomaly Detection and Long Short-Term
Memory Traffic Prediction"

coordinated by

Prof. Virgil DOBROTĂ

Professor Ovidiu POP

Dean

Professor Virgil DOBROTĂ

Chairman